Human review packet: CGGG20SelectiveInformationAcquisition

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: CGGG20SelectiveInformationAcquisition
Paper id: CGGG20SelectiveInformationAcquisition
Source version: AIES 2020 arXiv:1911.02715 source archive; canonical audit surface is the byte-pinned sample-sigconf.tex archive member
Source record: audit/paper_statement_map.json
Rows: 5
Generated: 2026-09-09
Source URL: official source

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper CGGG20SelectiveInformationAcquisition --serve
```

Open http://127.0.0.1:8765/ in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Decision.MonotoneExpectation

Declaration location: reusable library

Lean declaration body

type:
```
{ω : Type u_1} → ((ω → ℝ) → ℝ) → Prop
```

value:
```
fun {ω : Type u_1} (expect : (ω → ℝ) → ℝ) => ∀ (f g : ω → ℝ), (∀ (x : ω), f x ≤ g x) → expect f ≤ expect g
```

AppliedModelingLib.Decision.FiniteLinearExpectation

Declaration location: reusable library

Lean declaration body

type:
```
{ω : Type u_1} → ((ω → ℝ) → ℝ) → Prop
```

value:
```
fun {ω : Type u_1} (expect : (ω → ℝ) → ℝ) =>
  AppliedModelingLib.Decision.MonotoneExpectation expect ∧
    (∀ (c : ℝ) (f : ω → ℝ), (expect fun (x : ω) => c * f x) = c * expect f) ∧
      ∀ (f g : ω → ℝ), (expect fun (x : ω) => f x + g x) = expect f + expect g
```

Direct retained dependencies

- AppliedModelingLib.Decision.MonotoneExpectation

AppliedModelingLib.Probability.RealThreshold

Declaration location: reusable library

Lean declaration body

type:
```
Type
```

constructor AppliedModelingLib.Probability.RealThreshold.negInf:
```
AppliedModelingLib.Probability.RealThreshold
```

constructor AppliedModelingLib.Probability.RealThreshold.finite:
```
ℝ → AppliedModelingLib.Probability.RealThreshold
```

constructor AppliedModelingLib.Probability.RealThreshold.posInf:
```
AppliedModelingLib.Probability.RealThreshold
```

CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData

Declaration location: paper-local

Lean declaration body

field expect:
```
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome → (Outcome → ℝ) → ℝ
```

field expect_finiteLinear:
```
∀ {Applicant : Type u_1} {Outcome : Type u_2}
  (self : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome),
  AppliedModelingLib.Decision.FiniteLinearExpectation self.expect
```

field posteriorUtility:
```
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome → Applicant → Outcome → ℝ
```

field allocCost:
```
{Applicant : Type u_1} →
  {Outcome : Type u_2} → CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome → Applicant → ℝ
```

Direct retained dependencies

- AppliedModelingLib.Decision.FiniteLinearExpectation

CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy

Declaration location: paper-local

Lean declaration body

```
type:
Type u_3 → Type u_4 → Type (max u_3 u_4)

value:
fun (Applicant : Type u_3) (Outcome : Type u_4) => Applicant → Outcome → ℝ
```

CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.expectedCost

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [Fintype Applicant] →
      CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome →
        CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} [Fintype Applicant]
    (D : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome)
    (policy : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome) =>
  D.expect fun (outcome : Outcome) => ∑ item : Applicant, D.allocCost item * policy item outcome
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData
- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.expectedPosteriorUtility

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [Fintype Applicant] →
      CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome →
        CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} [Fintype Applicant]
    (D : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome)
    (policy : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome) =>
  D.expect fun (outcome : Outcome) => ∑ item : Applicant, policy item outcome * D.posteriorUtility item outcome
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData
- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureCostAwareData

Declaration location: paper-local

Lean declaration body

```
field law:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [inst : MeasurableSpace Outcome] →
      CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome → MeasureTheory.Measure Outcome

field posteriorUtility:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [inst : MeasurableSpace Outcome] →
      CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome → Applicant → Outcome → ℝ

field posteriorUtility_measurable:
∀ {Applicant : Type u_1} {Outcome : Type u_2} [inst : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome) (item : Applicant),
  Measurable (self.posteriorUtility item)

field posteriorUtility_integrable:
∀ {Applicant : Type u_1} {Outcome : Type u_2} [inst : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome) (item : Applicant),
  MeasureTheory.Integrable (self.posteriorUtility item) self.law

field allocCost:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [inst : MeasurableSpace Outcome] →
      CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome → Applicant → ℝ
```

CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy

Declaration location: paper-local

Lean declaration body

```
type:
Type u_3 → Type u_4 → Type (max u_3 u_4)

value:
fun (Applicant : Type u_3) (Outcome : Type u_4) => Applicant → Outcome → ℝ
```

CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyBounds

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome → Prop

value:
fun {Applicant : Type u_1} {Outcome : Type u_2}
    (policy : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome) =>
  ∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [MeasurableSpace Outcome] →
      CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome → Prop

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} [MeasurableSpace Outcome]
    (policy : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome) =>
  ∀ (item : Applicant), Measurable (policy item)
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.expectedCost

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [Fintype Applicant] →
      [inst : MeasurableSpace Outcome] →
        CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome →
          CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome)
    (policy : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome) =>
  ∫ (outcome : Outcome), ∑ item : Applicant, D.allocCost item * policy item outcome ∂D.law
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.expectedPosteriorUtility

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    [Fintype Applicant] →
      [inst : MeasurableSpace Outcome] →
        CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome →
          CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome)
    (policy : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome) =>
  ∫ (outcome : Outcome), ∑ item : Applicant, policy item outcome * D.posteriorUtility item outcome ∂D.law
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData

Declaration location: paper-local

Lean declaration body

```
field toMeasureCostAwareData:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : MeasurableSpace Outcome] →
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group →
          CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome

field group:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : MeasurableSpace Outcome] →
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group → Applicant → Group
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData

CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy

Declaration location: paper-local

Lean declaration body

```
type:
Type u_4 → Type u_5 → Type (max u_4 u_5)

value:
fun (Applicant : Type u_4) (Outcome : Type u_5) =>
  CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_4} →
  {Outcome : Type u_5} →
    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome → Prop

value:
fun {Applicant : Type u_4} {Outcome : Type u_5}
    (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
  CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyBounds policy
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyBounds
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_4} →
  {Outcome : Type u_5} →
    [MeasurableSpace Outcome] →
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome → Prop

value:
fun {Applicant : Type u_4} {Outcome : Type u_5} [MeasurableSpace Outcome]
    (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
  CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable policy
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.expectedCost

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [Fintype Applicant] →
        [inst : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group →
            CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group)
    (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
  D.expectedCost policy
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.expectedCost
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy

CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} → {Outcome : Type u_2} → (Applicant → Outcome → ℝ) → (Applicant → Outcome → ℝ) → Prop

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} (posteriorUtility policy : Applicant → Outcome → ℝ) =>
  ∀ (item : Applicant),
    Function.FactorsThrough (policy item) fun (outcome : Outcome) (applicant : Applicant) =>
      posteriorUtility applicant outcome
```

CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
AppliedModelingLib.Probability.RealThreshold
```

Direct retained dependencies

- AppliedModelingLib.Probability.RealThreshold

CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data

Declaration location: paper-local

Lean declaration body

```
type:
Type u_1 → (Outcome : Type u_2) → Type u_3 → [MeasurableSpace Outcome] → Type (max (max u_1 u_2) u_3)

value:
fun (Applicant : Type u_1) (Outcome : Type u_2) (Group : Type u_3) [MeasurableSpace Outcome] =>
  CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData

CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_groupwise_threshold_policy

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [Fintype Applicant] →
        [inst : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome
              Group →
            (Group → CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold) →
              (Group → ℝ) →
                CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome →
                  Prop

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome Group)
```

```
  (threshold : Group → CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold)
  (alpha : Group → ℝ)
  (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
(∀ (g : Group), 0 ≤ alpha g ∧ alpha g ≤ 1) ∧
  ∀ (item : Applicant) (outcome : Outcome),
    match threshold (D.group item) with
    | AppliedModelingLib.Probability.RealThreshold.negInf => policy item outcome = 1
    | AppliedModelingLib.Probability.RealThreshold.finite t =>
      (t < D.posteriorUtility item outcome / D.allocCost item → policy item outcome = 1) ∧
        (D.posteriorUtility item outcome / D.allocCost item = t → policy item outcome = alpha (D.group item)) ∧
          (D.posteriorUtility item outcome / D.allocCost item < t → policy item outcome = 0)
    | AppliedModelingLib.Probability.RealThreshold.posInf => policy item outcome = 0
```

Direct retained dependencies

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data

CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [Fintype Applicant] →
        [inst : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome
            Group →
            (Group → AppliedModelingLib.Probability.RealThreshold) →
              (Group → ℝ) →
                CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome →
                  Prop
```

```
value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome Group)
    (threshold : Group → AppliedModelingLib.Probability.RealThreshold) (alpha : Group → ℝ)
    (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
  (∀ (g : Group), 0 ≤ alpha g ∧ alpha g ≤ 1) ∧
    ∀ (item : Applicant) (outcome : Outcome),
      match threshold (D.group item) with
      | AppliedModelingLib.Probability.RealThreshold.negInf => policy item outcome = 1
      | AppliedModelingLib.Probability.RealThreshold.finite t =>
        (t < D.posteriorUtility item outcome → policy item outcome = 1) ∧
          (D.posteriorUtility item outcome = t → policy item outcome = alpha (D.group item)) ∧
            (D.posteriorUtility item outcome < t → policy item outcome = 0)
      | AppliedModelingLib.Probability.RealThreshold.posInf => policy item outcome = 0
```

Direct retained dependencies

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PostScreeningAllocationRule

Declaration location: paper-local

Lean declaration body

```
type:
Type u_4 → Type u_4
```

```
value:
fun (Applicant : Type u_4) => (Applicant → ℝ) → Applicant → ℝ
```

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} → CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant → Prop

value:
fun {Applicant : Type u_1} (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) =>
  ∀ (item : Applicant), 0 ≤ p.probability item ∧ p.probability item ≤ 1
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

CGGG20SelectiveInformationAcquisition.postScreeningEstimateVector

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : MeasurableSpace Outcome] →
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group →
          Outcome → Applicant → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group) (outcome : Outcome)
    (a : Applicant) =>
  D.posteriorUtility a outcome
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData

CGGG20SelectiveInformationAcquisition.postScreeningEstimateSigma

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : MeasurableSpace Outcome] →
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group →
          MeasurableSpace Outcome

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group) =>
  MeasurableSpace.comap (CGGG20SelectiveInformationAcquisition.postScreeningEstimateVector D) inferInstance
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.postScreeningEstimateVector

CGGG20SelectiveInformationAcquisition.full_vector_posterior_sufficiency

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : MeasurableSpace Outcome] →
        (CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant →
           CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group) →
          (Applicant → Outcome → ℝ) → Prop
```

```
value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [MeasurableSpace Outcome]
    (D :
      CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant →
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group)
    (actualUtility : Applicant → Outcome → ℝ) =>
  ∀ (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) (item : Applicant),
    (D p).posteriorUtility item =ᵐ[(D p).law]
      (D p).law[actualUtility item | CGGG20SelectiveInformationAcquisition.postScreeningEstimateSigma (D p)]
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy
- CGGG20SelectiveInformationAcquisition.postScreeningEstimateSigma

CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data

Declaration location: paper-local

Lean declaration body

```
type:
(Applicant : Type u_1) →
  (Outcome : Type u_2) → Type u_3 → [Fintype Applicant] → [MeasurableSpace Outcome] → Type (max (max u_1 u_2) u_3)
```

```
value:
fun (Applicant : Type u_1) (Outcome : Type u_2) (Group : Type u_3) [Fintype Applicant] [MeasurableSpace Outcome] =>
  CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData

CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group →
            Type u_1
```

```
value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group) =>
  CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair D
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          {D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group} →
            D.PolicyPair → Applicant → Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    {D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group} (pair : D.PolicyPair)
    (a : Applicant) (a_1 : Outcome) =>
  D.evaluatePostScreeningAllocationRule pair.screening pair.allocationRule a a_1
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.evaluatePostScreeningAllocationRule

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend

Declaration location: paper-local

Lean declaration body

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group →
            CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
    (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) =>
  ∑ item : Applicant, D.screenCost item * p.probability item
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Paper-specific semantic prerequisites

CGGG20SelectiveInformationAcquisition.PaperInterface.definition1_threshold_policySpec

Paper-local declaration:
CGGG20SelectiveInformationAcquisition.PaperInterface.definition1_threshold_policySpec
Source locator: cited publication, lines 366-376

Verbatim paper-source connection

```
\begin{defn}[Threshold Policy]
   A \emph{threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1, \ldots, m\),
   ↪  such that
      \begin{equation*}
         A_i = \begin{cases}
            1 & \hat U_i > t_{g_i},\\
            \alpha_{g_i} & \hat U_i = t_{g_i}, \\
            0 & \hat U_i < t_{g_i},
         \end{cases}
      \end{equation*}
   where \(g_i\) denotes the group membership of individual \(i\).
\end{defn}
```

[Next verbatim source excerpt]

```
\begin{defn}[Threshold Policy]
   A \emph{threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1, \ldots, m\),
   ↪  such that
      \begin{equation*}
         A_i = \begin{cases}
            1 & \hat U_i > t_{g_i},\\
            \alpha_{g_i} & \hat U_i = t_{g_i}, \\
            0 & \hat U_i < t_{g_i},
         \end{cases}
      \end{equation*}
   where \(g_i\) denotes the group membership of individual \(i\).
\end{defn}
```

Lean-expanded paper semantic target

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [Fintype Applicant] →
        [Fintype Group] →
          [inst : MeasurableSpace Outcome] →
            CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome
              Group →
              (Group → AppliedModelingLib.Probability.RealThreshold) →
                (Group → ℝ) →
                  CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome →
                    Prop

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [Fintype Group]
    [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome Group)
    (threshold : Group → AppliedModelingLib.Probability.RealThreshold) (alpha : Group → ℝ)
    (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
  (∀ (g : Group), 0 ≤ alpha g ∧ alpha g ≤ 1) ∧
    ∀ (item : Applicant) (outcome : Outcome),
      match threshold (D.group item) with
      | AppliedModelingLib.Probability.RealThreshold.negInf => policy item outcome = 1
      | AppliedModelingLib.Probability.RealThreshold.finite t =>
        (t < D.posteriorUtility item outcome → policy item outcome = 1) ∧
          (D.posteriorUtility item outcome = t → policy item outcome = alpha (D.group item)) ∧
            (D.posteriorUtility item outcome < t → policy item outcome = 0)
      | AppliedModelingLib.Probability.RealThreshold.posInf => policy item outcome = 0
```

Direct retained dependencies

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data

Recorded source-to-declaration screening **Verdict:** matches

Reason: The pinned Definition 1 fixes a group threshold in the extended reals and a group tie probability in [0,1]. The expanded predicate gives allocation 1 strictly above the posterior-utility threshold, alpha at equality, and 0 strictly below it, for every applicant and outcome. RealThreshold has exactly negative infinity, finite real and positive infinity constructors, and their displayed branches give the required all-allocate and never-allocate policies. The group map,

posterior utility and allocation-function carriers are explicit; threshold and alpha are exposed witnesses for the parameterized definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

CGGG20SelectiveInformationAcquisition.PaperInterface.definition3_cost_aware_threshold_policySpec

Paper-local declaration:
CGGG20SelectiveInformationAcquisition.PaperInterface.definition3_cost_aware_threshold_policySpec
Source locator: cited publication, lines 626-636

Verbatim paper-source connection

```
\begin{defn}[Cost-Aware Threshold Policy]
   A \emph{cost-aware threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1,
   ↪  \ldots, m\), such that
      \begin{equation*}
         A_i = \begin{cases}
            1 & \hat U_i / c_i > t_{g_i},\\
            \alpha_{g_i} & \hat U_i / c_i = t_{g_i}, \\
            0 & \hat U_i / c_i < t_{g_i},
         \end{cases}
      \end{equation*}
   where \(g_i\) denotes the group membership of individual \(i\) and \(c_i\) denotes the cost of allocating resources to individual \(i\).
\end{defn}
```

Lean-expanded paper semantic target

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [Fintype Applicant] →
        [Fintype Group] →
          [inst : MeasurableSpace Outcome] →
            (D :
               CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome
                 Group) →
              (∀ (item : Applicant), 0 < D.allocCost item) →
                (Group → CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold) →
                  (Group → ℝ) →
                    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant
                      Outcome →
                      Prop

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [Fintype Group]
    [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data Applicant Outcome Group)
    (hcost_pos : ∀ (item : Applicant), 0 < D.allocCost item)
    (threshold : Group → CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold)
    (alpha : Group → ℝ)
    (policy : CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy Applicant Outcome) =>
  (∀ (g : Group), 0 ≤ alpha g ∧ alpha g ≤ 1) ∧
    ∀ (item : Applicant) (outcome : Outcome),
      match threshold (D.group item) with
      | AppliedModelingLib.Probability.RealThreshold.negInf => policy item outcome = 1
      | AppliedModelingLib.Probability.RealThreshold.finite t =>
        (t < D.posteriorUtility item outcome / D.allocCost item → policy item outcome = 1) ∧
          (D.posteriorUtility item outcome / D.allocCost item = t → policy item outcome = alpha (D.group item)) ∧
            (D.posteriorUtility item outcome / D.allocCost item < t → policy item outcome = 0)
      | AppliedModelingLib.Probability.RealThreshold.posInf => policy item outcome = 0
```

Direct retained dependencies

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_measure_group_allocation_data

Recorded source-to-declaration screening **Verdict:** matches

Reason: The pinned Definition 3 has the same groupwise threshold rule applied to posterior utility divided by applicant allocation cost. The actual predicate uses precisely that ratio in all three comparisons, restricts every alpha to [0,1], and handles both infinite thresholds correctly. Its strictly positive cost parameter is exactly the bound ordinary clarification CGGG20-STRICT-POSITIVE-ALLOCATION-COST-01, so this is an archival match under that approved reading, not a materially corrected target. The displayed aliases and carriers preserve the applicant costs, group assignments and outcome-dependent utilities.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Paper-local declaration:
CGGG20SelectiveInformationAcquisition.ScreeningPolicy
Source locator: cited publication, lines 250-323

Verbatim paper-source connection

We assume the value of lending to an applicant i is given by the random variable U_i.
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a government agency.
In general, the lender has only partial information about U_i. More specifically, if the lender chooses not to screen an applicant, we assume the lender knows only the applicant's conditional expectation $\mu_i = \EE [U_i \mid X_i = x_i]$
given their pre-screening covariates x_i, such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn $\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]$, where $\tilde{x}_i$ denotes the additional information one gains through screening.
For example, $\tilde{x}_i$ might encode applicant i's history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of $\C D = (D_1, \ldots, D_n)$, where $D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]$.
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost $\cs$ for screening an applicant
and a cost $\ca$ for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
$\C P = (p_1, \dots, p_n)$, meaning that each applicant i is selected to be screened independently with probability p_i.
Let $\C S = (S_1, \dots, S_n)$ indicate which applicants are ultimately screened under this policy; therefore, $S_i \in \{0,1\}$ is a Bernoulli random variable with probability of success p_i.

Given this randomized screening policy, we can now write the information $\hat {\C U} = (\hat U_1, \ldots, \hat U_n)$ the lender has at the end of the screening phase as follows:

$$\hat U_i = \begin{cases} \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\ \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1. \end{cases}$$

In other words, $\hat U_i$ is the lender's post-screening estimated utility of giving a loan to applicant i.
In particular, if $S_i = 1$ (i.e., the applicant is screened), the lender's estimate changes from $\EE[U_i\mid X_i = x_i]$,
the estimate based only on applicant i's pre-screening covariates x_i,
to
$\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] $, which incorporates the post-screening information $\tilde{x}_i$.

Finally, the lender chooses an allocation policy, denoted $\C A(\hat {\C U}) = (A_1, \dots A_n)$,
where $A_i \in [0,1]$
specifies the probability a loan is (independently) offered to each applicant.
Importantly, $\C A$ is a function of
the lender's post-screening utility estimates
$\hat {\C U}$.
For example, the lender might give loans to the individuals with the highest post-screening utility estimates, up to the budget constraint.

Combining all of the above, the lender's optimization problem is to choose screening and allocation policies $(\C P^*, \C A^*)$ that maximize expected welfare,

$$(\C P^*, \C A^*) \in \argmax_{\C P, \C A} \bbE\left[\sum_{i = 1}^n U_i A_i\right],$$

subject to being budget-balanced in expectation,%
\footnote{By requiring the budget constraint to hold only in expectation---rather than exactly---we are able to find a computationally tractable solution to the problem. In practice, such a constraint means that agencies are allowed some flexibility as long as they don't overspend on average, which, we believe, is often a realistic requirement.
}

$$\EE \left[\sum_{i = 1}^n \cs S_i + \ca A_i \right] \leq B,$$

where B is a fixed, non-negative constant.

In some settings, decision makers may value diversity in their allocations.
For example, they may wish to ensure a certain minimum number of loans are provided to groups that historically have been excluded from credit markets.
One can encode this policy preference directly into the utilities, in which case the resulting optimization problem would incorporate one's value for diversity.
That approach, however, requires decision makers to agree upon these utilities to interpret the results, which can be challenging.

Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into m groups $\{G_1, \ldots, G_m\}$;
for example, if $m = 2$, we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy $(\C{P}^*, \C{A}^*)$
to allocate at least $\Lambda_j \geq 0$ utility to group G_j, where these utilities do not themselves include any value for diversity:

$$\EE \left[\sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j.$$

In practice, as we discuss below, one would solve this optimization problem for a range of Λ, which traces out the Pareto frontier of possible policies across different group constraints, corresponding to different values for diversity.

[Next verbatim source excerpt]

Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and
↪ \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
\label{eq:key}
\EE \left[\sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[\sum_{i = 1}^n c_i \Delta_i \right].
\end{equation}

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Lean-elaborated paper signature

```
field probability:
{Applicant : Type u_1} → CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant → Applicant → ℝ

field probability_bounds:
∀ {Applicant : Type u_1} (self : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) (item : Applicant),
  0 ≤ self.probability item ∧ self.probability item ≤ 1
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: Compared the ScreeningPolicy target and every displayed support with the source's P = (p_1,...,p_n), where applicant i is screened independently with probability p_i. The structure quantifies one real probability per applicant and requires 0 <= p_i <= 1. The material ScreeningAllocationData prerequisite supplies the missing stochastic realization exactly: for every policy and complete bit vector it gives the Bernoulli product probability, together with the approved exogeneity and policy-invariant latent-law conditions. All displayed paper uses pass policies through that data, repeat ScreeningPolicyBounds, and compute expected screening expenditure as sum_i screenCost_i p_i, matching E[sum_i screenCost_i S_i]. Independence is therefore not lost by being represented in the post-screening law rather than duplicated in the probability-vector structure, and no displayed use broadens the source policy domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData

Paper-local declaration:
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
Source locator: cited publication, lines 250-323

Verbatim paper-source connection

We assume the value of lending to an applicant \(i\) is given by the random variable \(U_i\).
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a government agency.
In general, the lender has only partial information about \(U_i\). More specifically, if the lender chooses not to screen an applicant, we assume the lender knows only the applicant's conditional expectation \(\mu_i = \EE [U_i \mid X_i = x_i]\)
given their pre-screening covariates \(x_i\), such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn \(\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]\), where \(\tilde{x}_i\) denotes the additional information one gains through screening.
For example, \(\tilde{x}_i\) might encode applicant \(i\)'s history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of \(\C D = (D_1, \ldots, D_n)\), where \(D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]\).
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost \(\cs\) for screening an applicant
and a cost \(\ca\) for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
\(\C P = (p_1, \dots, p_n)\), meaning that each applicant \(i\) is selected to be screened independently with probability \(p_i\).
Let \(\C S = (S_1, \dots, S_n)\) indicate which applicants are ultimately screened under this policy; therefore, \(S_i \in \{0,1\}\) is a Bernoulli random variable with probability of success \(p_i\).

Given this randomized screening policy, we can now write the information \(\hat {\C U} = (\hat U_1, \ldots, \hat U_n)\) the lender has at the end of the screening phase as follows:

$$\hat U_i = \begin{cases} \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\ \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1. \end{cases}$$

In other words, \(\hat U_i\) is the lender's post-screening estimated utility of giving a loan to applicant \(i\).
In particular, if \(S_i = 1\) (i.e., the applicant is screened), the lender's estimate changes from \(\EE[U_i\mid X_i = x_i]\),
the estimate based only on applicant \(i\)'s pre-screening covariates \(x_i\),
to
\(\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] \), which incorporates the post-screening information \(\tilde{x}_i\).

Finally, the lender chooses an allocation policy, denoted \(\C A(\hat {\C U}) = (A_1, \dots A_n)\),
where \(A_i \in [0,1]\)
specifies the probability a loan is (independently) offered to each applicant.
Importantly, \(\C A\) is a function of
the lender's post-screening utility estimates
\(\hat {\C U}\).
For example, the lender might give loans to the individuals with the highest post-screening utility estimates, up to the budget constraint.

Combining all of the above, the lender's optimization problem is to choose screening and allocation policies \((\C P^*, \C A^*)\) that maximize expected welfare,

$$(\C P^*, \C A^*) \in \argmax_{\C P, \C A} \bbE\left[\sum_{i = 1}^n U_i A_i\right], \tag{eq:def1}$$

subject to being budget-balanced in expectation,%
\footnote{By requiring the budget constraint to hold only in expectation---rather than exactly---we are able to find a computationally tractable solution to the problem. In practice, such a constraint means that agencies are allowed some flexibility as long as they don't overspend on average, which, we believe, is often a realistic requirement.
}

$$\EE \left[\sum_{i = 1}^n \cs S_i + \ca A_i \right] \leq B, \tag{eq:def2}$$

where \(B\) is a fixed, non-negative constant.

In some settings, decision makers may value diversity in their allocations.
For example, they may wish to ensure a certain minimum number of loans are provided to groups that historically have been excluded from credit markets.
One can encode this policy preference directly into the utilities, in which case the resulting optimization problem would incorporate one's value for diversity.
That approach, however, requires decision makers to agree upon these utilities to interpret the results, which can be challenging.

Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into \(m\) groups \(\{G_1, \ldots, G_m\}\);
for example, if \(m = 2\), we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy \((\C{P}^*, \C{A}^*)\)
to allocate at least \(\Lambda_j \geq 0\) utility to group \(G_j\), where these utilities do not themselves include any value for diversity:

$$\EE \left[\sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j. \tag{eq:def3}$$

In practice, as we discuss below, one would solve this optimization problem for a range of \(\Lambda\), which traces out the Pareto frontier of possible policies across different group constraints, corresponding to different values for diversity.

[Next verbatim source excerpt]

Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and ↪ \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
\label{eq:key}
\EE \left[\sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[\sum_{i = 1}^n c_i \Delta_i \right].
\end{equation}

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Lean-elaborated paper signature

```
field screenCost:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group → Applicant → ℝ

field allocCost:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group → Applicant → ℝ

field actualUtility:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group →
            Applicant → Outcome → ℝ

field post:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group →
            CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant →
              CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData Applicant Outcome Group

field screened:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group →
            Applicant → Outcome → Bool

field screened_measurable:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group) (item : Applicant),
  Measurable (self.screened item)

field priorUtility:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group → Applicant → ℝ

field screenedUtility:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group →
            Applicant → Outcome → ℝ

field posterior_utility_from_screening:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) (item : Applicant) (outcome : Outcome),
  (self.post p).posteriorUtility item outcome =
    if self.screened item outcome = true then self.screenedUtility item outcome else self.priorUtility item

field group:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group → Applicant → Group
```

```
field post_group:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant), (self.post p).group = self.group

field post_allocCost:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant), (self.post p).allocCost = self.allocCost

field post_probability:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant),
  MeasureTheory.IsProbabilityMeasure (self.post p).law

field screening_independent_bernoulli:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) (bits : Applicant → Bool),
  (self.post p).law {outcome : Outcome | ∀ (item : Applicant), self.screened item outcome = bits item} =
    ∏ item : Applicant, ENNReal.ofReal (if bits item = true then p.probability item else 1 - p.probability item)

field screening_exogenous_to_potential_utilities:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant),
  ProbabilityTheory.IndepFun (fun (outcome : Outcome) (item : Applicant) => self.screened item outcome)
    (fun (outcome : Outcome) (item : Applicant) => (self.actualUtility item outcome, self.screenedUtility item outcome))
    (self.post p).law

field potential_utility_law_invariant:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p p' : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant),
  MeasureTheory.Measure.map
      (fun (outcome : Outcome) (item : Applicant) =>
        (self.actualUtility item outcome, self.screenedUtility item outcome))
      (self.post p).law =
    MeasureTheory.Measure.map
      (fun (outcome : Outcome) (item : Applicant) =>
        (self.actualUtility item outcome, self.screenedUtility item outcome))
      (self.post p').law

field post_finite:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant), MeasureTheory.IsFiniteMeasure (self.post p).law

field actual_utility_integrable:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
  (p : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant) (item : Applicant),
  MeasureTheory.Integrable (self.actualUtility item) (self.post p).law

field posterior_utility_is_condExp_observed:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  (self : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group),
  CGGG20SelectiveInformationAcquisition.full_vector_posterior_sufficiency self.post self.actualUtility
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy
- CGGG20SelectiveInformationAcquisition.full_vector_posterior_sufficiency

Recorded source-to-declaration screening **Verdict:** matches

Reason: Compared the complete ScreeningAllocationData target and all displayed supports with screening_allocation_pair_model. The fields represent the finite applicant utilities, policy-indexed probability law, complete independent-Bernoulli screening vector with probabilities p_i, the source's prior-versus-screened definition of hat U_i, and groups and allocation costs fixed across screening policies. screening_exogenous_to_potential_utilities plus potential_utility_law_invariant implement exactly approved clarification CGGG20-EXOGENOUS-SCREENING-RANDOMIZATION-01; posterior_utility_is_condExp_observed, expanded through postScreeningEstimateSigma and postScreeningEstimateVector, implements exactly approved clarification CGGG20-FULL-VECTOR-POSTERIOR-SUFFICIENCY-01 and supports the displayed E[A_i U_i] = E[A_i hat U_i] route for A(hat U). Probability, measurability, finiteness, and integrability make the source expectations well-defined. The three displayed claim uses impose the source allocation bounds and measurability, budget and group constraints, and either the fixed main-text costs or the displayed cost-aware domain. No binder, formula, admissible paper-domain restriction, or conclusion differs from the source under the approved readings.

Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair

Paper-local declaration:
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair
Source locator: cited publication, lines 250-323

Verbatim paper-source connection

We assume the value of lending to an applicant i is given by the random variable U_i.
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a government agency.
In general, the lender has only partial information about U_i. More specifically, if the lender chooses not to screen an applicant, we assume the lender knows only the applicant's conditional expectation $\mu_i = \EE [U_i \mid X_i = x_i]$
given their pre-screening covariates x_i, such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn $\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]$, where $\tilde{x}_i$ denotes the additional information one gains through screening.
For example, $\tilde{x}_i$ might encode applicant i's history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of $\C D = (D_1, \ldots, D_n)$, where $D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]$.
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost $\cs$ for screening an applicant
and a cost $\ca$ for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
$\C P = (p_1, \dots, p_n)$, meaning that each applicant i is selected to be screened independently with probability p_i.
Let $\C S = (S_1, \dots, S_n)$ indicate which applicants are ultimately screened under this policy; therefore, $S_i \in \{0,1\}$ is a Bernoulli random variable with probability of success p_i.

Given this randomized screening policy, we can now write the information $\hat {\C U} = (\hat U_1, \ldots, \hat U_n)$ the lender has at the end of the screening phase as follows:

$$\hat U_i = \begin{cases} \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\ \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1. \end{cases}$$

In other words, $\hat U_i$ is the lender's post-screening estimated utility of giving a loan to applicant i.
In particular, if $S_i = 1$ (i.e., the applicant is screened), the lender's estimate changes from $\EE[U_i\mid X_i = x_i]$,
the estimate based only on applicant i's pre-screening covariates x_i,
to
$\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] $, which incorporates the post-screening information $\tilde{x}_i$.

Finally, the lender chooses an allocation policy, denoted $\C A(\hat {\C U}) = (A_1, \dots A_n)$,
where $A_i \in [0,1]$
specifies the probability a loan is (independently) offered to each applicant.
Importantly, $\C A$ is a function of
the lender's post-screening utility estimates
$\hat {\C U}$.
For example, the lender might give loans to the individuals with the highest post-screening utility estimates, up to the budget constraint.

Combining all of the above, the lender's optimization problem is to choose screening and allocation policies $(\C P^*, \C A^*)$ that maximize expected welfare,

$$(\C P^*, \C A^*) \in \argmax_{\C P, \C A} \bbE\left[\sum_{i = 1}^n U_i A_i\right], \tag{eq:def1}$$

subject to being budget-balanced in expectation,%
\footnote{By requiring the budget constraint to hold only in expectation---rather than exactly---we are able to find a computationally tractable solution to the problem. In practice, such a constraint means that agencies are allowed some flexibility as long as they don't overspend on average, which, we believe, is often a realistic requirement.
}

$$\EE \left[\sum_{i = 1}^n \cs S_i + \ca A_i \right] \leq B, \tag{eq:def2}$$

where B is a fixed, non-negative constant.

In some settings, decision makers may value diversity in their allocations.
For example, they may wish to ensure a certain minimum number of loans are provided to groups that historically have been excluded from credit markets.
One can encode this policy preference directly into the utilities, in which case the resulting optimization problem would incorporate one's value for diversity.
That approach, however, requires decision makers to agree upon these utilities to interpret the results, which can be challenging.

Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into m groups $\{G_1, \ldots, G_m\}$;
for example, if $m = 2$, we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy $(\C{P}^*, \C{A}^*)$
to allocate at least $\Lambda_j \geq 0$ utility to group G_j, where these utilities do not themselves include any value for diversity:

$$\EE \left[\sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j. \tag{eq:def3}$$

In practice, as we discuss below, one would solve this optimization problem for a range of Λ, which traces out the Pareto frontier of possible policies across different group constraints, corresponding to different values for diversity.

[Next verbatim source excerpt]

Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and ↪ \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
\label{eq:key}
\EE \left[\sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[\sum_{i = 1}^n c_i \Delta_i \right].
\end{equation}

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Lean-elaborated paper signature

```
field screening:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          {D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group} →
            D.PolicyPair → CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant

field allocationRule:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          {D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group} →
            D.PolicyPair →
              CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PostScreeningAllocationRule Applicant

field allocationRule_measurable:
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome]
  {D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group} (self : D.PolicyPair)
  (item : Applicant), Measurable fun (estimates : Applicant → ℝ) => self.allocationRule estimates item
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PostScreeningAllocationRule
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Recorded source-to-declaration screening **Verdict:** matches

Reason: Compared the PolicyPair fields with the source's joint choice (P,A), especially the verbatim requirement that A is a function of the entire post-screening estimate vector hat U and that each A_i is a loan probability. The expanded target contains one ScreeningPolicy and a coordinatewise measurable rule (Applicant -> real) -> Applicant -> real; PolicyPair.allocation and evaluatePostScreeningAllocationRule show that its only run-time input is exactly the full posteriorUtility vector. In every displayed theorem use, every original, competitor, and replacement evaluated allocation is explicitly measurable and bounded in [0,1], while the screening law and approved posterior/exogeneity readings come from D; the replacement also keeps the original screening policy. Thus the reusable pair type's real-valued generality is restricted to the exact source allocation domain at every paper use, with no arbitrary Outcome-dependent allocation or changed optimization quantifier.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.evaluatePostScreeningAllocationRule

Paper-local declaration:
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.evaluatePostScreeningAllocationRule
Source locator: cited publication, lines 250-323

Verbatim paper-source connection

We assume the value of lending to an applicant i is given by the random variable U_i.
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a government agency.
In general, the lender has only partial information about U_i. More specifically, if the lender chooses not to screen an applicant, we assume the lender knows only the applicant's conditional expectation $\mu_i = \EE [U_i \mid X_i = x_i]$
given their pre-screening covariates x_i, such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn $\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]$, where $\tilde{x}_i$ denotes the additional information one gains through screening.
For example, $\tilde{x}_i$ might encode applicant i's history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of $\C D = (D_1, \ldots, D_n)$, where $D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]$.
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost $\cs$ for screening an applicant
and a cost $\ca$ for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
$\C P = (p_1, \dots, p_n)$, meaning that each applicant i is selected to be screened independently with probability p_i.
Let $\C S = (S_1, \dots, S_n)$ indicate which applicants are ultimately screened under this policy; therefore, $S_i \in \{0,1\}$ is a Bernoulli random variable with probability of success p_i.

Given this randomized screening policy, we can now write the information $\hat {\C U} = (\hat U_1, \ldots, \hat U_n)$ the lender has at the end of the screening phase as follows:

$$\hat U_i = \begin{cases} \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\ \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1. \end{cases}$$

In other words, $\hat U_i$ is the lender's post-screening estimated utility of giving a loan to applicant i.
In particular, if $S_i = 1$ (i.e., the applicant is screened), the lender's estimate changes from $\EE[U_i\mid X_i = x_i]$,
the estimate based only on applicant i's pre-screening covariates x_i,
to
$\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] $, which incorporates the post-screening information $\tilde{x}_i$.

Finally, the lender chooses an allocation policy, denoted $\C A(\hat {\C U}) = (A_1, \dots A_n)$,
where $A_i \in [0,1]$
specifies the probability a loan is (independently) offered to each applicant.
Importantly, $\C A$ is a function of
the lender's post-screening utility estimates
$\hat {\C U}$.
For example, the lender might give loans to the individuals with the highest post-screening utility estimates, up to the budget constraint.

Combining all of the above, the lender's optimization problem is to choose screening and allocation policies $(\C P^*, \C A^*)$ that maximize expected welfare,

$$(\C P^*, \C A^*) \in \argmax_{\C P, \C A} \bbE\left[\sum_{i = 1}^n U_i A_i\right], \tag{eq:def1}$$

subject to being budget-balanced in expectation,%
\footnote{By requiring the budget constraint to hold only in expectation---rather than exactly---we are able to find a computationally tractable solution to the problem. In practice, such a constraint means that agencies are allowed some flexibility as long as they don't overspend on average, which, we believe, is often a realistic requirement.
}

$$\EE \left[\sum_{i = 1}^n \cs S_i + \ca A_i \right] \leq B, \tag{eq:def2}$$

where B is a fixed, non-negative constant.

In some settings, decision makers may value diversity in their allocations.
For example, they may wish to ensure a certain minimum number of loans are provided to groups that historically have been excluded from credit markets.
One can encode this policy preference directly into the utilities, in which case the resulting optimization problem would incorporate one's value for diversity.
That approach, however, requires decision makers to agree upon these utilities to interpret the results, which can be challenging.

Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into m groups $\{G_1, \ldots, G_m\}$;
for example, if $m = 2$, we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy $(\C{P}^*, \C{A}^*)$
to allocate at least $\Lambda_j \geq 0$ utility to group G_j, where these utilities do not themselves include any value for diversity:

$$\EE \left[\sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j. \tag{eq:def3}$$

In practice, as we discuss below, one would solve this optimization problem for a range of Λ, which traces out the Pareto frontier of possible policies across different group constraints, corresponding to different values for diversity.

[Next verbatim source excerpt]

```
Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and
↪  \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
  \label{eq:key}
    \EE \left[ \sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[ \sum_{i = 1}^n c_i \Delta_i \right].
  \end{equation}
```

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Lean-expanded paper semantic target

```
type:
{Applicant : Type u_1} →
  {Outcome : Type u_2} →
    {Group : Type u_3} →
      [inst : Fintype Applicant] →
        [inst_1 : MeasurableSpace Outcome] →
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group →
            CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant →
              CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PostScreeningAllocationRule Applicant →
                Applicant → Outcome → ℝ

value:
fun {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [Fintype Applicant] [MeasurableSpace Outcome]
    (D : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData Applicant Outcome Group)
    (screening : CGGG20SelectiveInformationAcquisition.ScreeningPolicy Applicant)
    (rule : CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PostScreeningAllocationRule Applicant)
    (a : Applicant) (a_1 : Outcome) =>
  rule (fun (applicant : Applicant) => (D.post screening).posteriorUtility applicant a_1) a
```

Direct retained dependencies

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PostScreeningAllocationRule
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Recorded source-to-declaration screening

Verdict: matches
Reason: Compared the fully expanded value with the source sentence A(hat U) = (A_1,...,A_n), where A is a function of the lender's complete post-screening utility-estimate vector. The definition evaluates rule at (fun applicant => (D.post screening).posteriorUtility applicant outcome) and then selects the requested applicant coordinate, exactly yielding A_i(hat U) without any additional outcome input. PolicyPair.allocation uses this evaluator, and all three displayed claim uses impose the source [0,1] and measurability conditions on the resulting allocation and keep the same screening in the threshold replacement. The approved full-vector posterior and exogenous-screening records are carried by the displayed D prerequisite. No argument, formula, domain, or use-site conclusion is changed.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 385-388

Verbatim source input

```
\begin{thm}
\label{thm:main}
    Suppose the constrained optimization problem defined by Eqs.~\eqref{eq:def1}, \eqref{eq:def2}, and \eqref{eq:def3} has a solution \((\C P^*, \C
    ↪  A^*)\). Then there is a threshold policy \(\C T^*\) such that \((\C P^*, \C T^*)\) is also a solution.
\end{thm}

[Next verbatim source excerpt]

We assume the value of lending to an applicant \(i\) is given by the random variable \(U_i\).
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a
↪  government agency.
In general, the lender has only partial information about \(U_i\). More specifically, if the lender chooses not to screen an applicant, we assume the lender
↪  knows only the applicant's conditional expectation \(\mu_i = \EE [U_i \mid X_i = x_i]\)
given their pre-screening covariates \(x_i\), such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn \(\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]\), where \(\tilde{x}_i\) denotes the
↪  additional information one gains through screening.
For example, \(\tilde{x}_i\) might encode applicant \(i\)'s history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of
↪  creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of \(\C D = (D_1, \ldots, D_n)\), where \(D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]\).
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the
↪  available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost \(\cs\) for screening an applicant
and a cost \(\ca\) for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online
↪  appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
\(\C P = (p_1, \dots, p_n)\), meaning that each applicant \(i\) is selected to be screened independently with probability \(p_i\).
Let \(\C S = (S_1, \dots, S_n)\) indicate which applicants are ultimately screened under this policy; therefore, \(S_i \in \{0,1\}\) is a Bernoulli random variable
↪  with probability of success \(p_i\).

Given this randomized screening policy, we can now write the information \(\hat {\C U} = (\hat U_1, \ldots, \hat U_n)\) the lender has at the end of the
↪  screening phase as follows:
    \begin{equation}
        \hat U_i = \begin{cases}
            \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\
            \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1.
        \end{cases}
    \end{equation}
In other words, \(\hat U_i\) is the lender's post-screening estimated utility of giving a loan to applicant \(i\).
In particular, if \(S_i = 1\) (i.e., the applicant is screened), the lender's estimate changes from \(\EE[U_i\mid X_i = x_i]\),
the estimate based only on applicant \(i\)'s pre-screening covariates \(x_i\),
to
\(\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] \), which incorporates the post-screening information \(\tilde{x}_i\).

Finally, the lender chooses an allocation policy, denoted \(\C A(\hat {\C U}) = (A_1, \dots A_n)\),
where \(A_i \in [0,1]\)
specifies the probability a loan is (independently) offered to each applicant.
Importantly, \(\C A\) is a function of
the lender's post-screening utility estimates
\(\hat {\C U}\).

[Next verbatim source excerpt]

Combining all of the above, the lender's optimization problem is to choose screening and allocation policies \((\C P^*, \C A^*)\) that maximize expected
↪  welfare,
\begin{equation}
\label{eq:def1}
    (\C P^*, \C A^*) \in \argmax_{\C P, \C A} \bbE\left[\sum_{i = 1}^n U_i A_i\right],
\end{equation}
subject to being budget-balanced in expectation,%
\footnote{By requiring the budget constraint to hold only in expectation---rather than exactly---we are able to find a computationally tractable solution to
↪  the problem. In practice, such a constraint means that agencies are allowed some flexibility as long as they don't overspend on average,
which, we believe, is often a realistic requirement.
}

\begin{equation}
\label{eq:def2}
    \EE \left[ \sum_{i = 1}^n \cs S_i + \ca  A_i \right] \leq B,
\end{equation}
where \(B\) is a fixed, non-negative constant.

In some settings, decision makers may value diversity in their allocations.
For example, they may wish to ensure a certain minimum number of loans are provided to groups that historically have been excluded from credit markets.
One can encode this policy preference directly into the utilities, in which case the resulting optimization problem would incorporate one's value for diversity.
That approach, however, requires decision makers to agree upon these utilities to interpret the results, which can be challenging.

Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into \(m\) groups \(\{G_1, \ldots, G_m\}\);
for example, if \(m = 2\), we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy \((\C{P}^*, \C{A}^*)\)
```

to allocate at least \(\Lambda_j \geq 0\) utility to group \(G_j\), where these utilities do not themselves include any value for diversity:
\begin{equation}
\label{eq:def3}
\EE \left[\sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j.
\end{equation}
In practice, as we discuss below, one would solve this optimization problem for a range of \(\Lambda\), which traces out the Pareto frontier of possible policies across different group constraints, corresponding to different values for diversity.

[Next verbatim source excerpt]

Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
\label{eq:key}
\EE \left[\sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[\sum_{i = 1}^n c_i \Delta_i \right].
\end{equation}

[Next verbatim source excerpt]

\begin{defn}[Threshold Policy]
A \emph{threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1, \ldots, m\), such that
\begin{equation*}
A_i = \begin{cases}
1 & \hat U_i > t_{g_i},\\
\alpha_{g_i} & \hat U_i = t_{g_i}, \\
0 & \hat U_i < t_{g_i},
\end{cases}
\end{equation*}
where \(g_i\) denotes the group membership of individual \(i\).
\end{defn}

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Expanded PaperInterface specification

```
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome] [Fintype Group]
  (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group)
  (screeningCost : ℝ),
  (∀ (item : Applicant), D.screenCost item = screeningCost) →
    ∀ (c : ℝ),
      0 < c →
        ∀ (budget : ℝ),
          0 ≤ budget →
            ∀ (diversityTarget : Group → ℝ),
              (∀ (g : Group), 0 ≤ diversityTarget g) →
                ∀ (original : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
                  (∀ (item : Applicant), D.allocCost item = c) →
                    ((CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
                          original.screening ∧
                        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                            original) ∧
                        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                            original) ∧
                        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                              original.screening +
                            (D.post original.screening).expectedCost
                              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                                original) ≤
                            budget ∧
                          ∀ (g : Group),
                            diversityTarget g ≤
                              ∫ (outcome : Outcome),
                                ∑ item : Applicant,
                                  if (D.post original.screening).group item = g then
                                    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                                      original item outcome *
                                      D.actualUtility item outcome
                                  else 0 ∂(D.post original.screening).law) ∧
                      ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
                        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
                            other.screening ∧
                          CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                              other) ∧
                          CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                              other) ∧
                          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                                other.screening +
                              (D.post other.screening).expectedCost
                                (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                                  other) ≤
                              budget ∧
                            ∀ (g : Group),
                              diversityTarget g ≤
                                ∫ (outcome : Outcome),
                                  ∑ item : Applicant,
                                    if (D.post other.screening).group item = g then
                                      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
```

```
                        other item outcome *
                      D.actualUtility item outcome
                    else 0 ∂(D.post other.screening).law) →
  ∫ (outcome : Outcome),
    ∑ item : Applicant,
      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
        other item outcome *
      D.actualUtility item outcome ∂(D.post other.screening).law ≤
   ∫ (outcome : Outcome),
    ∑ item : Applicant,
      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
        original item outcome *
      D.actualUtility item outcome ∂(D.post original.screening).law) →
∃ (threshold : Group → AppliedModelingLib.Probability.RealThreshold) (alpha : Group → ℝ)
 (replacement :
 CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
 replacement.screening = original.screening ∧
  CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy
    (D.post original.screening) threshold alpha
    (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
      replacement) ∧
   CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
     replacement.screening ∧
    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
        replacement) ∧
     CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
       (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
         replacement) ∧
      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
          original.screening +
        (D.post original.screening).expectedCost
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
            replacement) ≤
       budget ∧
       (∀ (g : Group),
         diversityTarget g ≤
          ∫ (outcome : Outcome),
            ∑ item : Applicant,
              if (D.post original.screening).group item = g then
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                  replacement item outcome *
                D.actualUtility item outcome
              else 0 ∂(D.post original.screening).law) ∧
        ∀
          (other :
            CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair
              D),
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
              other.screening ∧
            CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                other) ∧
             CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
               (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                 other) ∧
              CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend
                  D other.screening +
                (D.post other.screening).expectedCost
                  (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                    other) ≤
                budget ∧
               ∀ (g : Group),
                 diversityTarget g ≤
                  ∫ (outcome : Outcome),
                    ∑ item : Applicant,
                      if (D.post other.screening).group item = g then
                        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                          other item outcome *
                        D.actualUtility item outcome
                      else 0 ∂(D.post other.screening).law) →
            ∫ (outcome : Outcome),
              ∑ item : Applicant,
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                  other item outcome *
                D.actualUtility item outcome ∂(D.post other.screening).law ≤
             ∫ (outcome : Outcome),
              ∑ item : Applicant,
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                  replacement item outcome *
                D.actualUtility item outcome ∂(D.post original.screening).law
```

Retained governing declarations

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.expectedCost

- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Type u_3
4. parameter: Fintype Applicant
5. parameter: MeasurableSpace Outcome
6. parameter: Fintype Group
7. parameter: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group
8. parameter: ℝ
9. assumption: ∀ (item : Applicant), D.screenCost item = screeningCost
10. parameter: ℝ
11. assumption: 0 < c
12. parameter: ℝ
13. assumption: 0 ≤ budget
14. parameter: Group → ℝ
15. assumption: ∀ (g : Group), 0 ≤ diversityTarget g
16. parameter: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D
17. assumption: ∀ (item : Applicant), D.allocCost item = c
18. assumption: (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds original.screening ∧
    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D original.screening +
        (D.post original.screening).expectedCost
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ≤
      budget ∧
    ∀ (g : Group),
      diversityTarget g ≤
        ∫ (outcome : Outcome),
          ∑ item : Applicant,
            if (D.post original.screening).group item = g then
              CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original item
                outcome *
                D.actualUtility item outcome
            else 0 ∂(D.post original.screening).law) ∧
  ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
    (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D other.screening +
          (D.post other.screening).expectedCost
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ≤
        budget ∧
      ∀ (g : Group),
        diversityTarget g ≤
          ∫ (outcome : Outcome),
            ∑ item : Applicant,
              if (D.post other.screening).group item = g then
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
                  outcome *
                  D.actualUtility item outcome
              else 0 ∂(D.post other.screening).law) →
    ∫ (outcome : Outcome),
      ∑ item : Applicant,
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item outcome *
          D.actualUtility item outcome ∂(D.post other.screening).law ≤
      ∫ (outcome : Outcome),
        ∑ item : Applicant,
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original item outcome *
            D.actualUtility item outcome ∂(D.post original.screening).law
19. conclusion: ∃ (threshold : Group → AppliedModelingLib.Probability.RealThreshold) (alpha : Group → ℝ) (replacement :
  CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
  replacement.screening = original.screening ∧
    CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy
      (D.post original.screening) threshold alpha
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds replacement.screening ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D original.screening +
          (D.post original.screening).expectedCost
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ≤
```

```
  budget ∧
(∀ (g : Group),
   diversityTarget g ≤
     ∫ (outcome : Outcome),
       ∑ item : Applicant,
         if (D.post original.screening).group item = g then
           CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
             replacement item outcome *
           D.actualUtility item outcome
         else 0 ∂(D.post original.screening).law) ∧
 ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
   (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
          other) ∧
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
            other.screening +
          (D.post other.screening).expectedCost
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
              other) ≤
          budget ∧
         ∀ (g : Group),
          diversityTarget g ≤
            ∫ (outcome : Outcome),
              ∑ item : Applicant,
                if (D.post other.screening).group item = g then
                  CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                    other item outcome *
                  D.actualUtility item outcome
                else 0 ∂(D.post other.screening).law) →
    ∫ (outcome : Outcome),
      ∑ item : Applicant,
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
          outcome *
        D.actualUtility item outcome ∂(D.post other.screening).law ≤
     ∫ (outcome : Outcome),
       ∑ item : Applicant,
         CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
           replacement item outcome *
         D.actualUtility item outcome ∂(D.post original.screening).law
```

Lean proof endpoint (built separately)

CGGG20SelectiveInformationAcquisition.PaperInterface.theorem1_threshold_policies_suffice

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared theorem2 and Eqs. def1--def3 with the expanded target: an optimal feasible screening/allocation pair under nonnegative expected budget and nonnegative group lower bounds is replaced at the same screening policy by groupwise posterior-utility RealThreshold allocation, remains feasible, and is optimal against all feasible policy pairs. The prerequisites give the source’s common screening and strictly positive common allocation costs, bounded measurable full-vector allocation rules, independent exogenous screening law, and realized-utility integrals under each induced post-screening law, so the quantified model domain, objective, constraints, and conclusion agree.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 643-650

Verbatim source input

```
\begin{lem}
\label{lem:main}
    Let \(\C T\) be a cost-aware threshold policy with a single threshold \(t > 0\), and suppose \(\C T\) has an expected cost \(C\)---i.e., \(\sum_i \EE [c_i \cdot
    ↪  T_i] = C\)---and expected utility \(\Upsilon\)---i.e., \(\sum_i \EE [U_i \cdot T_i] = \Upsilon\). Let \(\C A\) be any other allocation policy.
    \begin{description}
        \item[Case 1:] If the expected cost of \(\C A\) is \(C\), then the expected utility of \(\C A\) is less than or equal to \(\Upsilon\).
        \item[Case 2:] If the expected utility of \(\C A\) is \(\Upsilon\), then the expected cost of \(\C A\) is greater than or equal to \(C\).
    \end{description}
\end{lem}
```

[Next verbatim source excerpt]

```
\begin{defn}[Cost-Aware Threshold Policy]
    A \emph{cost-aware threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1,
    ↪  \ldots, m\), such that
        \begin{equation*}
            A_i = \begin{cases}
                1 & \hat U_i / c_i > t_{g_i},\\
                \alpha_{g_i} & \hat U_i / c_i = t_{g_i}, \\
                0 & \hat U_i / c_i < t_{g_i},
            \end{cases}
        \end{equation*}
    where \(g_i\) denotes the group membership of individual \(i\) and \(c_i\) denotes the cost of allocating resources to individual \(i\).
\end{defn}
```

[Next verbatim source excerpt]

```
We assume the value of lending to an applicant \(i\) is given by the random variable \(U_i\).
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a
↪  government agency.
In general, the lender has only partial information about \(U_i\). More specifically, if the lender chooses not to screen an applicant, we assume the lender
↪  knows only the applicant's conditional expectation \(\mu_i = \EE [U_i \mid X_i = x_i]\)
given their pre-screening covariates \(x_i\), such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn \(\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]\), where \(\tilde{x}_i\) denotes the
↪  additional information one gains through screening.
For example, \(\tilde{x}_i\) might encode applicant \(i\)'s history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of
↪  creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of \(\C D = (D_1, \ldots, D_n)\), where \(D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]\).
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the
↪  available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost \(\cs\) for screening an applicant
and a cost \(\ca\) for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online
↪  appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
\(\C P = (p_1, \dots, p_n)\), meaning that each applicant \(i\) is selected to be screened independently with probability \(p_i\).
Let \(\C S = (S_1, \dots, S_n)\) indicate which applicants are ultimately screened under this policy; therefore, \(S_i \in \{0,1\}\) is a Bernoulli random variable
↪  with probability of success \(p_i\).

Given this randomized screening policy, we can now write the information \(\hat {\C U} = (\hat U_1, \ldots, \hat U_n)\) the lender has at the end of the
↪  screening phase as follows:
    \begin{equation}
        \hat U_i = \begin{cases}
            \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\
            \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1.
        \end{cases}
    \end{equation}
In other words, \(\hat U_i\) is the lender's post-screening estimated utility of giving a loan to applicant \(i\).
In particular, if \(S_i = 1\) (i.e., the applicant is screened), the lender's estimate changes from \(\EE[U_i\mid X_i = x_i]\),
the estimate based only on applicant \(i\)'s pre-screening covariates \(x_i\),
to
\(\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] \), which incorporates the post-screening information \(\tilde{x}_i\).

Finally, the lender chooses an allocation policy, denoted \(\C A(\hat {\C U}) = (A_1, \dots A_n)\),
where \(A_i \in [0,1]\)
specifies the probability a loan is (independently) offered to each applicant.
Importantly, \(\C A\) is a function of
the lender's post-screening utility estimates
\(\hat {\C U}\).
```

[Next verbatim source excerpt]

```
Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and
↪  \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
    \label{eq:key}
        \EE \left[ \sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[ \sum_{i = 1}^n c_i \Delta_i \right].
    \end{equation}
```

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information

about an applicant’s utility.

Expanded PaperInterface specification

```
(∀ {Applicant : Type u_1} {Outcome : Type u_2} [inst : Fintype Applicant]
   (D : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome)
   (actualUtility : Applicant → Outcome → ℝ),
   (∀ (policy : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
        policy →
       (∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1) →
        (D.expect fun (outcome : Outcome) => ∑ item : Applicant, policy item outcome * actualUtility item outcome) =
          D.expectedPosteriorUtility policy) →
     ∀ (t alpha : ℝ)
       (threshold other :
         CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
       (∀ (item : Applicant), 0 < D.allocCost item) →
         0 < t →
           (0 ≤ alpha ∧
              alpha ≤ 1 ∧
               ∀ (item : Applicant) (outcome : Outcome),
                 (t < D.posteriorUtility item outcome / D.allocCost item → threshold item outcome = 1) ∧
                   (D.posteriorUtility item outcome / D.allocCost item = t → threshold item outcome = alpha) ∧
                     (D.posteriorUtility item outcome / D.allocCost item < t → threshold item outcome = 0)) →
             (∀ (item : Applicant) (outcome : Outcome), 0 ≤ other item outcome ∧ other item outcome ≤ 1) →
               CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior
                  D.posteriorUtility other →
                 D.expectedCost threshold = D.expectedCost other →
                   (D.expect fun (outcome : Outcome) =>
                      ∑ item : Applicant, other item outcome * actualUtility item outcome) ≤
                     D.expect fun (outcome : Outcome) =>
                       ∑ item : Applicant, threshold item outcome * actualUtility item outcome) ∧
 ∀ {Applicant : Type u_3} {Outcome : Type u_4} [inst : Fintype Applicant]
   (D : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome)
   (actualUtility : Applicant → Outcome → ℝ),
   (∀ (policy : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
        policy →
       (∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1) →
        (D.expect fun (outcome : Outcome) => ∑ item : Applicant, policy item outcome * actualUtility item outcome) =
          D.expectedPosteriorUtility policy) →
     ∀ (t alpha : ℝ)
       (threshold other :
         CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
       (∀ (item : Applicant), 0 < D.allocCost item) →
         0 < t →
           (0 ≤ alpha ∧
              alpha ≤ 1 ∧
               ∀ (item : Applicant) (outcome : Outcome),
                 (t < D.posteriorUtility item outcome / D.allocCost item → threshold item outcome = 1) ∧
                   (D.posteriorUtility item outcome / D.allocCost item = t → threshold item outcome = alpha) ∧
                     (D.posteriorUtility item outcome / D.allocCost item < t → threshold item outcome = 0)) →
             (∀ (item : Applicant) (outcome : Outcome), 0 ≤ other item outcome ∧ other item outcome ≤ 1) →
               CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior
                  D.posteriorUtility other →
                 ((D.expect fun (outcome : Outcome) =>
                     ∑ item : Applicant, threshold item outcome * actualUtility item outcome) =
                   D.expect fun (outcome : Outcome) =>
                     ∑ item : Applicant, other item outcome * actualUtility item outcome) →
                 D.expectedCost threshold ≤ D.expectedCost other
```

Retained governing declarations

- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData
- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy
- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.expectedCost
- CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.expectedPosteriorUtility
- CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior

Lean-elaborated claim roles

1. conclusion:
```
(∀ {Applicant : Type u_1} {Outcome : Type u_2} [inst : Fintype Applicant]
   (D : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome)
   (actualUtility : Applicant → Outcome → ℝ),
   (∀ (policy : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
        policy →
       (∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1) →
        (D.expect fun (outcome : Outcome) => ∑ item : Applicant, policy item outcome * actualUtility item outcome) =
          D.expectedPosteriorUtility policy) →
     ∀ (t alpha : ℝ)
       (threshold other :
         CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
       (∀ (item : Applicant), 0 < D.allocCost item) →
         0 < t →
           (0 ≤ alpha ∧
              alpha ≤ 1 ∧
               ∀ (item : Applicant) (outcome : Outcome),
```

```
            (t < D.posteriorUtility item outcome / D.allocCost item → threshold item outcome = 1) ∧
              (D.posteriorUtility item outcome / D.allocCost item = t → threshold item outcome = alpha) ∧
                (D.posteriorUtility item outcome / D.allocCost item < t → threshold item outcome = 0)) →
          (∀ (item : Applicant) (outcome : Outcome), 0 ≤ other item outcome ∧ other item outcome ≤ 1) →
            CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior
                D.posteriorUtility other →
              D.expectedCost threshold = D.expectedCost other →
                (D.expect fun (outcome : Outcome) =>
                    ∑ item : Applicant, other item outcome * actualUtility item outcome) ≤
                  D.expect fun (outcome : Outcome) =>
                    ∑ item : Applicant, threshold item outcome * actualUtility item outcome) ∧
∀ {Applicant : Type u_3} {Outcome : Type u_4} [inst : Fintype Applicant]
  (D : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData Applicant Outcome)
  (actualUtility : Applicant → Outcome → ℝ),
  (∀ (policy : CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
        policy →
      (∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1) →
        (D.expect fun (outcome : Outcome) => ∑ item : Applicant, policy item outcome * actualUtility item outcome) =
          D.expectedPosteriorUtility policy) →
    ∀ (t alpha : ℝ)
      (threshold other :
        CGGG20SelectiveInformationAcquisition.ExpectedCostAwareData.AllocationPolicy Applicant Outcome),
      (∀ (item : Applicant), 0 < D.allocCost item) →
        0 < t →
          (0 ≤ alpha ∧
              alpha ≤ 1 ∧
                ∀ (item : Applicant) (outcome : Outcome),
                  (t < D.posteriorUtility item outcome / D.allocCost item → threshold item outcome = 1) ∧
                    (D.posteriorUtility item outcome / D.allocCost item = t → threshold item outcome = alpha) ∧
                      (D.posteriorUtility item outcome / D.allocCost item < t → threshold item outcome = 0)) →
            (∀ (item : Applicant) (outcome : Outcome), 0 ≤ other item outcome ∧ other item outcome ≤ 1) →
              CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior
                  D.posteriorUtility other →
                ((D.expect fun (outcome : Outcome) =>
                    ∑ item : Applicant, threshold item outcome * actualUtility item outcome) =
                  D.expect fun (outcome : Outcome) =>
                    ∑ item : Applicant, other item outcome * actualUtility item outcome) →
                D.expectedCost threshold ≤ D.expectedCost other
```

Lean proof endpoint (built separately)

CGGG20SelectiveInformationAcquisition.PaperInterface.lemma1_cost_aware_threshold_non_domination_realizes_spec

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared both cases of lemma4 with the two expanded conjuncts: for a positive finite cost-ratio threshold (including [0,1] boundary randomization) versus any bounded full-posterior allocation, equal expected cost implies no greater realized expected utility, while equal realized expected utility implies no lower expected cost. The prerequisite definitions expand expected cost as the expectation of the finite sum of c_i A_i, and the displayed actual-to-posterior bridge plus the approved full-vector-posterior and strictly-positive-cost readings matches the source model.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 704-712

Verbatim source input

```
\begin{lem}
\label{lem:full_range}
    Let \(\Upsilon > 0\) be an achievable expected utility---that is, there is an allocation policy \(\C A\) such that \(\EE[\sum_i A_i U_i] = \Upsilon\)---and let
    ↪  \(C\) be an achievable expected cost.
    Then,
        \begin{itemize}
            \item There exists a cost-aware threshold policy \(\C T\) of expected utility \(\Upsilon\); and,
            \item There exists a (possibly different) cost-aware threshold policy \(\C T\) of expected cost \(C\).
        \end{itemize}
\end{lem}
```

[Next verbatim source excerpt]

```
\begin{defn}[Cost-Aware Threshold Policy]
    A \emph{cost-aware threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1,
    ↪  \ldots, m\), such that
        \begin{equation*}
            A_i = \begin{cases}
                1 & \hat U_i / c_i > t_{g_i},\\
                \alpha_{g_i} & \hat U_i / c_i = t_{g_i}, \\
                0 & \hat U_i / c_i < t_{g_i},
            \end{cases}
        \end{equation*}
    where \(g_i\) denotes the group membership of individual \(i\) and \(c_i\) denotes the cost of allocating resources to individual \(i\).
\end{defn}
```

[Next verbatim source excerpt]

```
We assume the value of lending to an applicant \(i\) is given by the random variable \(U_i\).
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a
↪  government agency.
In general, the lender has only partial information about \(U_i\). More specifically, if the lender chooses not to screen an applicant, we assume the lender
↪  knows only the applicant's conditional expectation \(\mu_i = \EE [U_i \mid X_i = x_i]\)
given their pre-screening covariates \(x_i\), such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn \(\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]\), where \(\tilde{x}_i\) denotes the
↪  additional information one gains through screening.
For example, \(\tilde{x}_i\) might encode applicant \(i\)'s history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of
↪  creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of \(\C D = (D_1, \ldots, D_n)\), where \(D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]\).
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the
↪  available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost \(\cs\) for screening an applicant
and a cost \(\ca\) for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online
↪  appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
\(\C P = (p_1, \dots, p_n)\), meaning that each applicant \(i\) is selected to be screened independently with probability \(p_i\).
Let \(\C S = (S_1, \dots, S_n)\) indicate which applicants are ultimately screened under this policy; therefore, \(S_i \in \{0,1\}\) is a Bernoulli random variable
↪  with probability of success \(p_i\).

Given this randomized screening policy, we can now write the information \(\hat {\C U} = (\hat U_1, \ldots, \hat U_n)\) the lender has at the end of the
↪  screening phase as follows:
    \begin{equation}
        \hat U_i = \begin{cases}
            \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\
            \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1.
        \end{cases}
    \end{equation}
In other words, \(\hat U_i\) is the lender's post-screening estimated utility of giving a loan to applicant \(i\).
In particular, if \(S_i = 1\) (i.e., the applicant is screened), the lender's estimate changes from \(\EE[U_i\mid X_i = x_i]\),
the estimate based only on applicant \(i\)'s pre-screening covariates \(x_i\),
to
\(\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] \), which incorporates the post-screening information \(\tilde{x}_i\).

Finally, the lender chooses an allocation policy, denoted \(\C A(\hat {\C U}) = (A_1, \dots A_n)\),
where \(A_i \in [0,1]\)
specifies the probability a loan is (independently) offered to each applicant.
Importantly, \(\C A\) is a function of
the lender's post-screening utility estimates
\(\hat {\C U}\).
```

[Next verbatim source excerpt]

```
Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and
↪  \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
    \label{eq:key}
        \EE \left[ \sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[ \sum_{i = 1}^n c_i \Delta_i \right].
    \end{equation}
```

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Expanded PaperInterface specification

```
(∀ {Applicant : Type u_1} {Outcome : Type u_2} [inst : Fintype Applicant] [inst_1 : MeasurableSpace Outcome]
   (D : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome)
   [MeasureTheory.IsFiniteMeasure D.law] (actualUtility : Applicant → Outcome → ℝ),
   (∀ (policy : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
        policy →
       (∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1) →
        ∫ (outcome : Outcome), ∑ item : Applicant, policy item outcome * actualUtility item outcome ∂D.law =
          D.expectedPosteriorUtility policy) →
     ∀ (Upsilon : ℝ),
      (∀ (item : Applicant), 0 < D.allocCost item) →
       0 < Upsilon →
        (∃ (original :
            CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
           CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable original ∧
            CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior
              D.posteriorUtility original ∧
             (∀ (item : Applicant) (outcome : Outcome), 0 ≤ original item outcome ∧ original item outcome ≤ 1) ∧
              ∫ (outcome : Outcome),
                 ∑ item : Applicant, original item outcome * actualUtility item outcome ∂D.law =
               Upsilon) →
         ∃ (threshold : AppliedModelingLib.Probability.RealThreshold) (alpha : ℝ) (policy :
           CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
           0 ≤ alpha ∧
            alpha ≤ 1 ∧
             (∀ (item : Applicant) (outcome : Outcome),
                match threshold, item, outcome with
                | AppliedModelingLib.Probability.RealThreshold.negInf, item, outcome => policy item outcome = 1
                | AppliedModelingLib.Probability.RealThreshold.finite t, item, outcome =>
                  (t < D.posteriorUtility item outcome / D.allocCost item → policy item outcome = 1) ∧
                    (D.posteriorUtility item outcome / D.allocCost item = t → policy item outcome = alpha) ∧
                      (D.posteriorUtility item outcome / D.allocCost item < t → policy item outcome = 0)
                | AppliedModelingLib.Probability.RealThreshold.posInf, item, outcome =>
                  policy item outcome = 0) ∧
              ∫ (outcome : Outcome),
                 ∑ item : Applicant, policy item outcome * actualUtility item outcome ∂D.law =
               Upsilon) ∧
∀ {Applicant : Type u_3} {Outcome : Type u_4} [inst : Fintype Applicant] [inst_1 : MeasurableSpace Outcome]
  (D : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome)
  [MeasureTheory.IsFiniteMeasure D.law] (C : ℝ),
  (∀ (item : Applicant), 0 < D.allocCost item) →
   (∃ (original : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable original ∧
       CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
         original ∧
        (∀ (item : Applicant) (outcome : Outcome), 0 ≤ original item outcome ∧ original item outcome ≤ 1) ∧
         D.expectedCost original = C) →
    ∃ (threshold : AppliedModelingLib.Probability.RealThreshold) (alpha : ℝ) (policy :
      CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
      0 ≤ alpha ∧
       alpha ≤ 1 ∧
        (∀ (item : Applicant) (outcome : Outcome),
           match threshold, item, outcome with
           | AppliedModelingLib.Probability.RealThreshold.negInf, item, outcome => policy item outcome = 1
           | AppliedModelingLib.Probability.RealThreshold.finite t, item, outcome =>
             (t < D.posteriorUtility item outcome / D.allocCost item → policy item outcome = 1) ∧
               (D.posteriorUtility item outcome / D.allocCost item = t → policy item outcome = alpha) ∧
                 (D.posteriorUtility item outcome / D.allocCost item < t → policy item outcome = 0)
           | AppliedModelingLib.Probability.RealThreshold.posInf, item, outcome => policy item outcome = 0) ∧
         D.expectedCost policy = C
```

Retained governing declarations

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.expectedCost
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.expectedPosteriorUtility
- CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior

Lean-elaborated claim roles

1. conclusion:
```
(∀ {Applicant : Type u_1} {Outcome : Type u_2} [inst : Fintype Applicant] [inst_1 : MeasurableSpace Outcome]
   (D : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome)
   [MeasureTheory.IsFiniteMeasure D.law] (actualUtility : Applicant → Outcome → ℝ),
   (∀ (policy : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
      CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
        policy →
```

```
      (∀ (item : Applicant) (outcome : Outcome), 0 ≤ policy item outcome ∧ policy item outcome ≤ 1) →
        ∫ (outcome : Outcome), ∑ item : Applicant, policy item outcome * actualUtility item outcome ∂D.law =
          D.expectedPosteriorUtility policy) →
    ∀ (Upsilon : ℝ),
      (∀ (item : Applicant), 0 < D.allocCost item) →
        0 < Upsilon →
          (∃ (original :
              CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
              CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable original ∧
                CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior
                    D.posteriorUtility original ∧
                  (∀ (item : Applicant) (outcome : Outcome), 0 ≤ original item outcome ∧ original item outcome ≤ 1) ∧
                    ∫ (outcome : Outcome),
                        ∑ item : Applicant, original item outcome * actualUtility item outcome ∂D.law =
                      Upsilon) →
            ∃ (threshold : AppliedModelingLib.Probability.RealThreshold) (alpha : ℝ) (policy :
              CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
              0 ≤ alpha ∧
                alpha ≤ 1 ∧
                  (∀ (item : Applicant) (outcome : Outcome),
                      match threshold, item, outcome with
                      | AppliedModelingLib.Probability.RealThreshold.negInf, item, outcome => policy item outcome = 1
                      | AppliedModelingLib.Probability.RealThreshold.finite t, item, outcome =>
                        (t < D.posteriorUtility item outcome / D.allocCost item → policy item outcome = 1) ∧
                          (D.posteriorUtility item outcome / D.allocCost item = t → policy item outcome = alpha) ∧
                            (D.posteriorUtility item outcome / D.allocCost item < t → policy item outcome = 0)
                      | AppliedModelingLib.Probability.RealThreshold.posInf, item, outcome =>
                        policy item outcome = 0) ∧
                    ∫ (outcome : Outcome),
                        ∑ item : Applicant, policy item outcome * actualUtility item outcome ∂D.law =
                      Upsilon) ∧
∀ {Applicant : Type u_3} {Outcome : Type u_4} [inst : Fintype Applicant] [inst_1 : MeasurableSpace Outcome]
  (D : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData Applicant Outcome)
  [MeasureTheory.IsFiniteMeasure D.law] (C : ℝ),
  (∀ (item : Applicant), 0 < D.allocCost item) →
    (∃ (original : CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
        CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicyMeasurable original ∧
          CGGG20SelectiveInformationAcquisition.PaperInterface.allocationPolicyUsesFullPosterior D.posteriorUtility
            original ∧
            (∀ (item : Applicant) (outcome : Outcome), 0 ≤ original item outcome ∧ original item outcome ≤ 1) ∧
              D.expectedCost original = C) →
      ∃ (threshold : AppliedModelingLib.Probability.RealThreshold) (alpha : ℝ) (policy :
        CGGG20SelectiveInformationAcquisition.MeasureCostAwareData.AllocationPolicy Applicant Outcome),
        0 ≤ alpha ∧
          alpha ≤ 1 ∧
            (∀ (item : Applicant) (outcome : Outcome),
                match threshold, item, outcome with
                | AppliedModelingLib.Probability.RealThreshold.negInf, item, outcome => policy item outcome = 1
                | AppliedModelingLib.Probability.RealThreshold.finite t, item, outcome =>
                  (t < D.posteriorUtility item outcome / D.allocCost item → policy item outcome = 1) ∧
                    (D.posteriorUtility item outcome / D.allocCost item = t → policy item outcome = alpha) ∧
                      (D.posteriorUtility item outcome / D.allocCost item < t → policy item outcome = 0)
                | AppliedModelingLib.Probability.RealThreshold.posInf, item, outcome => policy item outcome = 0) ∧
              D.expectedCost policy = C
```

Lean proof endpoint (built separately)

CGGG20SelectiveInformationAcquisition.PaperInterface.lemma2_full_range_cost_aware_threshold_attainment_realizes_spec

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared lemma5's two independent attainment claims with the expanded conjunction: every positive achievable realized expected utility has a cost-aware threshold policy attaining it, and every achievable expected cost has a possibly different such policy attaining it. Achievability witnesses are exactly bounded measurable allocations of the full posterior vector; RealThreshold supplies the finite and two endpoint cases, boundary probabilities lie in [0,1], expected cost is the displayed integral of c_i A_i, and the approved posterior-sufficiency and positive-cost readings account for the explicit Lean premises.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 735-757

Verbatim source input

We are now equipped to move to analyze settings in which there are multiple groups. We prove the following generalization of Theorem~\ref{thm:main}.

\begin{thm}
Suppose the constrained optimization problem defined by Eqs.~\eqref{eq:def1}, \eqref{eq:def2}, and \eqref{eq:def3} has a solution \((\C P^*, \C A^*)\).
Then there is a (not necessarily single-threshold) cost-aware threshold policy \(\C T^*\) such that \((\C P^*, \C T^*)\) is also a solution.
\end{thm}

\begin{proof}[Proof of Theorem \ref{thm:main}]
Consider a solution screening and allocation policy pair \((\C P^*, \C A^*)\) satisfying the allocation diversity constraints.

Note that each of the collections \(\C A_j^* = \{A_i^*\}_{i \in G_j}\) defines an allocation policy on the subpopulation \(G_j\).
Applying Lemma~\ref{lem:full_range} to each of them yields a threshold policy \(\C T_j^*\)---defined by the pair \((t_j, \alpha_j)\)---of the same expected cost as \(\C A_j^*\).
By Lemma~\ref{lem:main}, \(\C T_j^*\) achieves the same or higher expected utility on \(G_j\) as \(\C A_j^*\).

Let \(\C T^*\) be the threshold policy defined by thresholds \(t_1, \ldots, t_m\) and boundary randomization probabilities \(\alpha_1, \ldots, \alpha_m\).
The expected cost of \(\C T^*\) is the same as \(\C A^*\).
Moreover, \(\C T^*\) satisfies the group utility constraints, since the utility it achieves on each group \(G_j\) is greater than or equal to \(\C A^*\).
Lastly, the expected utility achieved by \(\C T^*\) is greater than or equal to that of \(\C A^*\).

Since \(\C A^*\) was assumed optimal, it must be that the expected utilities are equal. Therefore the pair \((\C P^*, \C T^*)\) is also a solution to the constrained optimization problem defined by Eqs.~\eqref{eq:def1}, \eqref{eq:def2}, and \eqref{eq:def3}.
\end{proof}

Theorem~\ref{thm:main} follows as an immediate corollary.

[Next verbatim source excerpt]

We assume the value of lending to an applicant \(i\) is given by the random variable \(U_i\).
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a government agency.
In general, the lender has only partial information about \(U_i\). More specifically, if the lender chooses not to screen an applicant, we assume the lender knows only the applicant's conditional expectation \(\mu_i = \EE [U_i \mid X_i = x_i]\)
given their pre-screening covariates \(x_i\), such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn \(\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]\), where \(\tilde{x}_i\) denotes the additional information one gains through screening.
For example, \(\tilde{x}_i\) might encode applicant \(i\)'s history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of \(\C D = (D_1, \ldots, D_n)\), where \(D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]\).
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost \(\cs\) for screening an applicant
and a cost \(\ca\) for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
\(\C P = (p_1, \dots, p_n)\), meaning that each applicant \(i\) is selected to be screened independently with probability \(p_i\).
Let \(\C S = (S_1, \dots, S_n)\) indicate which applicants are ultimately screened under this policy; therefore, \(S_i \in \{0,1\}\) is a Bernoulli random variable with probability of success \(p_i\).

Given this randomized screening policy, we can now write the information \(\hat {\C U} = (\hat U_1, \ldots, \hat U_n)\) the lender has at the end of the screening phase as follows:
\begin{equation}
\hat U_i = \begin{cases}
\EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\
\EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1.
\end{cases}
\end{equation}
In other words, \(\hat U_i\) is the lender's post-screening estimated utility of giving a loan to applicant \(i\).
In particular, if \(S_i = 1\) (i.e., the applicant is screened), the lender's estimate changes from \(\EE[U_i\mid X_i = x_i]\),
the estimate based only on applicant \(i\)'s pre-screening covariates \(x_i\),
to
\(\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] \), which incorporates the post-screening information \(\tilde{x}_i\).

Finally, the lender chooses an allocation policy, denoted \(\C A(\hat {\C U}) = (A_1, \dots A_n)\),
where \(A_i \in [0,1]\)
specifies the probability a loan is (independently) offered to each applicant.
Importantly, \(\C A\) is a function of
the lender's post-screening utility estimates
\(\hat {\C U}\).

[Next verbatim source excerpt]

We prove Theorem~\ref{thm:main} in a more general setting in which we allow the cost of screening \(\cs\) and cost of allocation \(\ca\) to vary for each individual.
In the special case of no screening (but with applicant-specific costs of allocation), we note that our optimization problem is equivalent to the fractional knapsack problem.
For fractional knapsack, the greedy strategy---in which one packs items in descending order of their value per weight---yields an optimal solution~\cite{dantzig1957discrete}.
In our case, we show that the same approach can be used to find an optimal allocation of loans once the set of applicants to screen has been appropriately selected.

To start, we generalize our definition of a threshold policy to account for the applicant-specific allocation costs.

[Next verbatim source excerpt]

```
Combining all of the above, the lender's optimization problem is to choose screening and allocation policies \((\C P^*, \C A^*)\) that maximize expected
↪  welfare,
\begin{equation}
\label{eq:def1}
  (\C P^*, \C A^*) \in \argmax_{\C P, \C A} \bbE\left[\sum_{i = 1}^n U_i A_i\right],
\end{equation}
subject to being budget-balanced in expectation,%
\footnote{By requiring the budget constraint to hold only in expectation---rather than exactly---we are able to find a computationally tractable solution to
↪  the problem. In practice, such a constraint means that agencies are allowed some flexibility as long as they don't overspend on average,
which, we believe, is often a realistic requirement.
}

\begin{equation}
\label{eq:def2}
  \EE \left[ \sum_{i = 1}^n \cs S_i + \ca  A_i \right] \leq B,
\end{equation}
where \(B\) is a fixed, non-negative constant.

In some settings, decision makers may value diversity in their allocations.
For example, they may wish to ensure a certain minimum number of loans are provided to groups that historically have been excluded from credit markets.
One can encode this policy preference directly into the utilities, in which case the resulting optimization problem would incorporate one's value for diversity.
That approach, however, requires decision makers to agree upon these utilities to interpret the results, which can be challenging.

Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into \(m\) groups \(\{G_1, \ldots, G_m\}\);
for example, if \(m = 2\), we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy \((\C{P}^*, \C{A}^*)\)
to allocate at least \(\Lambda_j \geq 0\) utility to group \(G_j\), where these utilities do not themselves include any value for diversity:
  \begin{equation}
  \label{eq:def3}
    \EE \left[ \sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j.
  \end{equation}
In practice, as we discuss below, one would solve this optimization problem for a range of \(\Lambda\), which traces out the Pareto frontier of possible
↪  policies across different group constraints, corresponding to different values for diversity.

[Next verbatim source excerpt]

\begin{defn}[Cost-Aware Threshold Policy]
  A \emph{cost-aware threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1,
  ↪  \ldots, m\), such that
    \begin{equation*}
      A_i = \begin{cases}
        1 & \hat U_i / c_i > t_{g_i},\\
        \alpha_{g_i} & \hat U_i / c_i = t_{g_i}, \\
        0 & \hat U_i / c_i < t_{g_i},
      \end{cases}
    \end{equation*}
  where \(g_i\) denotes the group membership of individual \(i\) and \(c_i\) denotes the cost of allocating resources to individual \(i\).
\end{defn}

[Next verbatim source excerpt]

Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and
↪  \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
\begin{equation}
  \label{eq:key}
    \EE \left[ \sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[ \sum_{i = 1}^n c_i \Delta_i \right].
  \end{equation}
```

Settled source reading The paper’s displayed estimates are read as the lender’s posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant’s estimate supplies no residual information about an applicant’s utility.

Expanded PaperInterface specification

```
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome] [Fintype Group]
  (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group)
  (budget : ℝ),
  0 ≤ budget →
    ∀ (diversityTarget : Group → ℝ),
      (∀ (g : Group), 0 ≤ diversityTarget g) →
        ∀ (original : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
          (∀ (item : Applicant), 0 < D.allocCost item) →
            ((CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds original.screening ∧
                CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                  (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
                CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                  (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                      original.screening +
                    (D.post original.screening).expectedCost
                      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        original) ≤
                  budget ∧
                ∀ (g : Group),
                  diversityTarget g ≤
                    ∫ (outcome : Outcome),
                      ∑ item : Applicant,
                        if (D.post original.screening).group item = g then
```

```
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                  original item outcome *
                D.actualUtility item outcome
              else 0 ∂(D.post original.screening).law) ∧
∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
 (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
     CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
       (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
         other) ∧
       CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
           other.screening +
         (D.post other.screening).expectedCost
           (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
             other) ≤
         budget ∧
        ∀ (g : Group),
         diversityTarget g ≤
           ∫ (outcome : Outcome),
             ∑ item : Applicant,
               if (D.post other.screening).group item = g then
                 CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                   other item outcome *
                 D.actualUtility item outcome
               else 0 ∂(D.post other.screening).law) →
   ∫ (outcome : Outcome),
     ∑ item : Applicant,
       CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
         outcome *
       D.actualUtility item outcome ∂(D.post other.screening).law ≤
    ∫ (outcome : Outcome),
      ∑ item : Applicant,
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original
          item outcome *
        D.actualUtility item outcome ∂(D.post original.screening).law) →
∃ (threshold :
  Group → CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold) (alpha :
  Group → ℝ) (replacement :
  CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
  replacement.screening = original.screening ∧
   CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_groupwise_threshold_policy
     (D.post original.screening) threshold alpha
     (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
       replacement) ∧
    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
      replacement.screening ∧
     CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
       (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
         replacement) ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
          replacement) ∧
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
            original.screening +
          (D.post original.screening).expectedCost
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
              replacement) ≤
          budget ∧
         (∀ (g : Group),
           diversityTarget g ≤
             ∫ (outcome : Outcome),
               ∑ item : Applicant,
                 if (D.post original.screening).group item = g then
                   CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                     replacement item outcome *
                   D.actualUtility item outcome
                 else 0 ∂(D.post original.screening).law) ∧
          ∀
            (other :
              CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
                other.screening ∧
              CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                  other) ∧
                CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                  (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                    other) ∧
                  CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                      other.screening +
                    (D.post other.screening).expectedCost
                      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        other) ≤
                    budget ∧
                   ∀ (g : Group),
                     diversityTarget g ≤
                       ∫ (outcome : Outcome),
                         ∑ item : Applicant,
                           if (D.post other.screening).group item = g then
                             CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                               other item outcome *
                             D.actualUtility item outcome
                           else 0 ∂(D.post other.screening).law) →
             ∫ (outcome : Outcome),
               ∑ item : Applicant,
                 CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                   other item outcome *
                 D.actualUtility item outcome ∂(D.post other.screening).law ≤
              ∫ (outcome : Outcome),
```

∑ item : Applicant,
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
replacement item outcome *
D.actualUtility item outcome ∂(D.post original.screening).law

Retained governing declarations

- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.expectedCost
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_groupwise_threshold_policy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Type u_3
4. parameter: Fintype Applicant
5. parameter: MeasurableSpace Outcome
6. parameter: Fintype Group
7. parameter: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group
8. parameter: ℝ
9. assumption: 0 ≤ budget
10. parameter: Group → ℝ
11. assumption: ∀ (g : Group), 0 ≤ diversityTarget g
12. parameter: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D
13. assumption: ∀ (item : Applicant), 0 < D.allocCost item
14. assumption: (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds original.screening ∧
CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D original.screening +
(D.post original.screening).expectedCost
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ≤
budget ∧
∀ (g : Group),
diversityTarget g ≤
∫ (outcome : Outcome),
∑ item : Applicant,
if (D.post original.screening).group item = g then
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original item
outcome *
D.actualUtility item outcome
else 0 ∂(D.post original.screening).law) ∧
∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D other.screening +
(D.post other.screening).expectedCost
(CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ≤
budget ∧
∀ (g : Group),
diversityTarget g ≤
∫ (outcome : Outcome),
∑ item : Applicant,
if (D.post other.screening).group item = g then
CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
outcome *
D.actualUtility item outcome
else 0 ∂(D.post other.screening).law) →
∫ (outcome : Outcome),
∑ item : Applicant,

```
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item outcome *
          D.actualUtility item outcome ∂(D.post other.screening).law ≤
      ∫ (outcome : Outcome),
        ∑ item : Applicant,
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original item outcome *
            D.actualUtility item outcome ∂(D.post original.screening).law
15. conclusion: ∃ (threshold : Group → CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_extended_threshold) (alpha :
  Group → ℝ) (replacement : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
  replacement.screening = original.screening ∧
    CGGG20SelectiveInformationAcquisition.ProofBridge.paper_cost_aware_groupwise_threshold_policy
      (D.post original.screening) threshold alpha
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
     CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds replacement.screening ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
       CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D original.screening +
            (D.post original.screening).expectedCost
              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ≤
          budget ∧
         (∀ (g : Group),
            diversityTarget g ≤
              ∫ (outcome : Outcome),
                ∑ item : Applicant,
                  if (D.post original.screening).group item = g then
                    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                      replacement item outcome *
                    D.actualUtility item outcome
                  else 0 ∂(D.post original.screening).law) ∧
          ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
           (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
              CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
               CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                  (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                    other) ∧
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                      other.screening +
                    (D.post other.screening).expectedCost
                      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        other) ≤
                  budget ∧
                 ∀ (g : Group),
                   diversityTarget g ≤
                     ∫ (outcome : Outcome),
                       ∑ item : Applicant,
                         if (D.post other.screening).group item = g then
                           CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                             other item outcome *
                           D.actualUtility item outcome
                         else 0 ∂(D.post other.screening).law) →
            ∫ (outcome : Outcome),
              ∑ item : Applicant,
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
                  outcome *
                D.actualUtility item outcome ∂(D.post other.screening).law ≤
             ∫ (outcome : Outcome),
               ∑ item : Applicant,
                 CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                   replacement item outcome *
                 D.actualUtility item outcome ∂(D.post original.screening).law
```

Lean proof endpoint (built separately)

CGGG20SelectiveInformationAcquisition.PaperInterface.appendix_theorem_cost_aware_threshold_policies_suffice

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared theorem6's optimal screening/allocation pair under the expected budget and group-utility lower bounds with the expanded conclusion: it keeps the original screening policy, replaces allocation by groupwise posterior-utility/allocation-cost RealThreshold rules with boundary probabilities in [0,1], remains feasible, and is optimal against every feasible policy pair. The displayed prerequisites implement applicant-specific expected allocation cost, arbitrary bounded measurable full-vector allocation rules, independent exogenous screening, and the approved full-vector-posterior and strictly-positive-cost readings, so the binders, domains, formulas, and conclusion agree.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 763-777

Verbatim source input

```
\begin{lem}
\label{lem:modest_gen}
    The conclusion of Theorem~\ref{thm:main} still holds even if the inequalities in Eq.~\eqref{eq:def3} are replaced with equalities, i.e., if the optimal
    ↪ policy pair \((\C P^*, \C A^*)\) satisfies
        \begin{equation}
            \EE \left[ \sum_{i \in G_j} U_i A_i \right] = \Lambda_j.
        \end{equation}
\end{lem}

\begin{proof}
    The proof is identical to that of Theorem~\ref{thm:main}, except that Lemma~\ref{lem:full_range} is used to construct a threshold policy \(\C T^*\)
    ↪ achieving exactly utility \(\Lambda_j\) on any constrained group \(G_j\).
    Case 2 of Lemma~\ref{lem:main} then shows that this utility is achieved at possibly less expected cost than \(\C A^*\).
    It follows immediately that the pair \((\C P^*, \C T^*)\) is a solution to the optimization problem.\footnote{
        In this case, it is actually possible that \(\C T^*\) will result in expected cost savings over \(\C A^*\), even though \(\C A^*\) is optimal. This can occur if
        ↪ there is no remaining positive utility to ``spend'' the savings on, since resources are already allocated at total expected cost less than \(B\) in all
        ↪ cases where \(\hat U_i\) is positive.
    }
\end{proof}
```

[Next verbatim source excerpt]

```
Here we take a complementary approach that explicitly allows value for diversity to differ across decision makers.
Suppose the applicant pool is partitioned into \(m\) groups \(\{G_1, \ldots, G_m\}\);
for example, if \(m = 2\), we might partition candidates into those who traditionally have had access to credit markets and those who have not.
Then we require the selected policy \((\C{P}^*, \C{A}^*)\)
to allocate at least \(\Lambda_j \geq 0\) utility to group \(G_j\), where these utilities do not themselves include any value for diversity:
    \begin{equation}
    \label{eq:def3}
        \EE \left[ \sum_{i \in G_j} U_i A_i \right] \geq \Lambda_j.
    \end{equation}
In practice, as we discuss below, one would solve this optimization problem for a range of \(\Lambda\), which traces out the Pareto frontier of possible
↪ policies across different group constraints, corresponding to different values for diversity.
```

[Next verbatim source excerpt]

```
We assume the value of lending to an applicant \(i\) is given by the random variable \(U_i\).
These utilities are intended to capture the full social value of providing loans, and we imagine the lender aims to optimize social welfare, as in the case of a
↪ government agency.
In general, the lender has only partial information about \(U_i\). More specifically, if the lender chooses not to screen an applicant, we assume the lender
↪ knows only the applicant's conditional expectation \(\mu_i = \EE [U_i \mid X_i = x_i]\)
given their pre-screening covariates \(x_i\), such as credit score for those applicants who have traditional credit histories.
On the other hand, if the lender opts to screen an applicant, they learn \(\EE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i]\), where \(\tilde{x}_i\) denotes the
↪ additional information one gains through screening.
For example, \(\tilde{x}_i\) might encode applicant \(i\)'s history of paying
their electricity or phone bills---information that is often feasible to acquire with some extra effort and which is a good indicator of
↪ creditworthiness~\cite{fdic2018}.

When deciding whom to screen, we assume
the lender knows the distribution of \(\C D = (D_1, \ldots, D_n)\), where \(D_i = \EE[U_i \mid X_i = x_i, \tilde{X}_i]\).
That is, the lender knows how their estimate of utility could change if they decide to screen each applicant, where these distributions may depend on the
↪ available pre-screening covariates.
A lender may, for example, thus choose only to screen applicants whose estimate is likely to substantially change given additional information.
We further assume the lender must pay a
fixed cost \(\cs\) for screening an applicant
and a cost \(\ca\) for underwriting a loan.
For simplicity we assume these costs do not vary across applicants, though it is straightforward to extend to the more general case; see the online
↪ appendix for details.

Based on knowledge of the above information and cost structure, the lender selects a randomized strategy to screen applicants.
That is, the lender chooses a vector
\(\C P = (p_1, \dots, p_n)\), meaning that each applicant \(i\) is selected to be screened independently with probability \(p_i\).
Let \(\C S = (S_1, \dots, S_n)\) indicate which applicants are ultimately screened under this policy; therefore, \(S_i \in \{0,1\}\) is a Bernoulli random variable
↪ with probability of success \(p_i\).

Given this randomized screening policy, we can now write the information \(\hat {\C U} = (\hat U_1, \ldots, \hat U_n)\) the lender has at the end of the
↪ screening phase as follows:
    \begin{equation}
        \hat U_i = \begin{cases}
            \EE[U_i \mid X_i = x_i] & \text{ if } S_i = 0, \\
            \EE[U_i \mid X_i = x_i, \tilde{X}_i] & \text{ if } S_i =1.
        \end{cases}
    \end{equation}
In other words, \(\hat U_i\) is the lender's post-screening estimated utility of giving a loan to applicant \(i\).
In particular, if \(S_i = 1\) (i.e., the applicant is screened), the lender's estimate changes from \(\EE[U_i\mid X_i = x_i]\),
the estimate based only on applicant \(i\)'s pre-screening covariates \(x_i\),
to
\(\bbE[U_i \mid X_i = x_i, \tilde{X}_i = \tilde{x}_i] \), which incorporates the post-screening information \(\tilde{x}_i\).

Finally, the lender chooses an allocation policy, denoted \(\C A(\hat {\C U}) = (A_1, \dots A_n)\),
where \(A_i \in [0,1]\)
specifies the probability a loan is (independently) offered to each applicant.
Importantly, \(\C A\) is a function of
the lender's post-screening utility estimates
\(\hat {\C U}\).
```

[Next verbatim source excerpt]

```
Now, since any allocation policy is by definition conditionally independent of \(U_i\) given \(\hat U_i\), it follows that \(\EE [A_i U_i] = \EE [A_i \hat U_i]\) and
↪ \(\EE [T_i U_i] = \EE [T_i \hat U_i]\). In consequence,
```

```
\begin{equation}
   \label{eq:key}
      \EE \left[ \sum_{i = 1}^n \Delta_i U_i \right] \geq t \cdot \EE \left[ \sum_{i = 1}^n c_i \Delta_i \right].
   \end{equation}
```

[Next verbatim source excerpt]

```
\begin{defn}[Threshold Policy]
   A \emph{threshold policy} is an allocation policy \(\C A\) for some fixed \(t_j \in \B R \cup \{-\infty, \infty\}\) and \(\alpha_j \in [0,1]\), \(j = 1, \ldots, m\),
   ↪  such that
      \begin{equation*}
         A_i = \begin{cases}
            1 & \hat U_i > t_{g_i},\\
            \alpha_{g_i} & \hat U_i = t_{g_i}, \\
            0 & \hat U_i < t_{g_i},
         \end{cases}
      \end{equation*}
   where \(g_i\) denotes the group membership of individual \(i\).
\end{defn}
```

Settled source reading The paper's displayed estimates are read as the lender's posterior utilities given the full post-screening estimate vector used by allocation; thus observing another applicant's estimate supplies no residual information about an applicant's utility.

Expanded PaperInterface specification

```
∀ {Applicant : Type u_1} {Outcome : Type u_2} {Group : Type u_3} [inst : Fintype Applicant]
  [inst_1 : MeasurableSpace Outcome] [Fintype Group]
  (D : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group)
  (screeningCost c : ℝ),
  0 < c →
    ∀ (budget : ℝ),
      0 ≤ budget →
        ∀ (equalityTarget : Group → ℝ)
          (original : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
          (∀ (item : Applicant), D.screenCost item = screeningCost) →
            (∀ (item : Applicant), D.allocCost item = c) →
              (∀ (g : Group), 0 ≤ equalityTarget g) →
                ((CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
                      original.screening ∧
                    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        original) ∧
                    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        original) ∧
                    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                          original.screening +
                        (D.post original.screening).expectedCost
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                            original) ≤
                      budget ∧
                    ∀ (g : Group),
                      ∫ (outcome : Outcome),
                        ∑ item : Applicant,
                          if (D.post original.screening).group item = g then
                            CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                              original item outcome *
                            D.actualUtility item outcome
                          else 0 ∂(D.post original.screening).law =
                      equalityTarget g) ∧
                  ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
                    (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
                          other.screening ∧
                        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                            other) ∧
                        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                            other) ∧
                        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                              other.screening +
                            (D.post other.screening).expectedCost
                              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                                other) ≤
                          budget ∧
                        ∀ (g : Group),
                          ∫ (outcome : Outcome),
                            ∑ item : Applicant,
                              if (D.post other.screening).group item = g then
                                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                                  other item outcome *
                                D.actualUtility item outcome
                              else 0 ∂(D.post other.screening).law =
                          equalityTarget g) →
                      ∫ (outcome : Outcome),
                        ∑ item : Applicant,
                          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other
                            item outcome *
                          D.actualUtility item outcome ∂(D.post other.screening).law ≤
                        ∫ (outcome : Outcome),
                          ∑ item : Applicant,
                            CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                              original item outcome *
                            D.actualUtility item outcome ∂(D.post original.screening).law) →
                    ∃ (threshold : Group → AppliedModelingLib.Probability.RealThreshold) (alpha : Group → ℝ) (replacement
```

```
: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
replacement.screening = original.screening ∧
  CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy
      (D.post original.screening) threshold alpha
      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
        replacement) ∧
    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
        replacement.screening ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
            replacement) ∧
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
              replacement) ∧
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
                original.screening +
              (D.post original.screening).expectedCost
                (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                  replacement) ≤
            budget ∧
           (∀ (g : Group),
              ∫ (outcome : Outcome),
                ∑ item : Applicant,
                  if (D.post original.screening).group item = g then
                    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                      replacement item outcome *
                    D.actualUtility item outcome
                  else 0 ∂(D.post original.screening).law =
                equalityTarget g) ∧
             ∀
              (other :
                CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair
                  D),
              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds
                  other.screening ∧
                CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                    (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                      other) ∧
                  CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                      (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        other) ∧
                    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend
                          D other.screening +
                        (D.post other.screening).expectedCost
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                            other) ≤
                      budget ∧
                     ∀ (g : Group),
                       ∫ (outcome : Outcome),
                         ∑ item : Applicant,
                           if (D.post other.screening).group item = g then
                             CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                               other item outcome *
                             D.actualUtility item outcome
                           else 0 ∂(D.post other.screening).law =
                         equalityTarget g) →
                 ∫ (outcome : Outcome),
                   ∑ item : Applicant,
                     CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                       other item outcome *
                     D.actualUtility item outcome ∂(D.post other.screening).law ≤
                  ∫ (outcome : Outcome),
                    ∑ item : Applicant,
                      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                        replacement item outcome *
                      D.actualUtility item outcome ∂(D.post original.screening).law
```

Retained governing declarations

- AppliedModelingLib.Probability.RealThreshold
- CGGG20SelectiveInformationAcquisition.MeasureCostAwareData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
- CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.expectedCost
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data
- CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds

- CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend
- CGGG20SelectiveInformationAcquisition.ScreeningPolicy

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Type u_3
4. parameter: Fintype Applicant
5. parameter: MeasurableSpace Outcome
6. parameter: Fintype Group
7. parameter: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_data Applicant Outcome Group
8. parameter: ℝ
9. parameter: ℝ
10. assumption: 0 < c
11. parameter: ℝ
12. assumption: 0 ≤ budget
13. parameter: Group → ℝ
14. parameter: CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D
15. assumption: ∀ (item : Applicant), D.screenCost item = screeningCost
16. assumption: ∀ (item : Applicant), D.allocCost item = c
17. assumption: ∀ (g : Group), 0 ≤ equalityTarget g
18. assumption: (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds original.screening ∧
    CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
        (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ∧
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D original.screening +
            (D.post original.screening).expectedCost
              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original) ≤
          budget ∧
        ∀ (g : Group),
          ∫ (outcome : Outcome),
            ∑ item : Applicant,
              if (D.post original.screening).group item = g then
                CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original item
                  outcome *
                D.actualUtility item outcome
              else 0 ∂(D.post original.screening).law =
            equalityTarget g) ∧
  ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
    (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D other.screening +
              (D.post other.screening).expectedCost
                (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ≤
            budget ∧
          ∀ (g : Group),
            ∫ (outcome : Outcome),
              ∑ item : Applicant,
                if (D.post other.screening).group item = g then
                  CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
                    outcome *
                  D.actualUtility item outcome
                else 0 ∂(D.post other.screening).law =
              equalityTarget g) →
      ∫ (outcome : Outcome),
        ∑ item : Applicant,
          CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item outcome *
            D.actualUtility item outcome ∂(D.post other.screening).law ≤
        ∫ (outcome : Outcome),
          ∑ item : Applicant,
            CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation original item outcome *
              D.actualUtility item outcome ∂(D.post original.screening).law
19. conclusion: ∃ (threshold : Group → AppliedModelingLib.Probability.RealThreshold) (alpha : Group → ℝ) (replacement :
    CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
    replacement.screening = original.screening ∧
      CGGG20SelectiveInformationAcquisition.ProofBridge.paper_main_text_groupwise_threshold_policy
          (D.post original.screening) threshold alpha
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds replacement.screening ∧
          CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
              (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
            CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ∧
              CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D original.screening +
                  (D.post original.screening).expectedCost
                    (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation replacement) ≤
                budget ∧
              (∀ (g : Group),
                  ∫ (outcome : Outcome),
                    ∑ item : Applicant,
                      if (D.post original.screening).group item = g then
                        CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                          replacement item outcome *
                        D.actualUtility item outcome
                      else 0 ∂(D.post original.screening).law =
                    equalityTarget g) ∧
                ∀ (other : CGGG20SelectiveInformationAcquisition.ProofBridge.paper_screening_allocation_pair D),
                  (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.ScreeningPolicyBounds other.screening ∧
                      CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyBounds
                          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other) ∧
                        CGGG20SelectiveInformationAcquisition.MeasureGroupAllocationData.AllocationPolicyMeasurable
                            (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
```

```
        other) ∧
      CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.screeningSpend D
          other.screening +
        (D.post other.screening).expectedCost
          (CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
            other) ≤
        budget ∧
       ∀ (g : Group),
        ∫ (outcome : Outcome),
          ∑ item : Applicant,
            if (D.post other.screening).group item = g then
              CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
                other item outcome *
              D.actualUtility item outcome
            else 0 ∂(D.post other.screening).law =
          equalityTarget g) →
∫ (outcome : Outcome),
  ∑ item : Applicant,
    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation other item
      outcome *
    D.actualUtility item outcome ∂(D.post other.screening).law ≤
 ∫ (outcome : Outcome),
  ∑ item : Applicant,
    CGGG20SelectiveInformationAcquisition.ScreeningAllocationData.PolicyPair.allocation
      replacement item outcome *
    D.actualUtility item outcome ∂(D.post original.screening).law
```

Lean proof endpoint (built separately)

CGGG20SelectiveInformationAcquisition.PaperInterface.lemma3_equality_diversity_threshold_sufficiency

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared lemma7's equality version of every group constraint with the full expanded policy-pair target: from an optimal feasible pair satisfying each group utility exactly, it produces a replacement with the same screening policy, groupwise posterior-utility thresholds and [0,1] boundary probabilities, the same equality targets, budget feasibility, and optimality against every equality-feasible pair. The common strictly positive allocation cost and the displayed exogenous-screening/full-vector-posterior prerequisites are precisely the approved source readings, and the objective and group integrals match the source formulas.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation